

RAK4631 / WisBlock Sensor Firmware — Operator Manual

What this technology does

EMU-01 is the physical LoRaWAN agriculture node. It uses a RAK4631 core on RAK19001, seven sensor types, the Arduino/nRF52 firmware family, OTAA, Class A, ADR, unconfirmed uplinks, and the project's 46-byte payload-v2.

This manual is for **EMU-01 only**. The separate security node uses RUI3; see [20-rui3-security-node.md](#).

Current commissioned baseline

Role	EMU-01
FQBN	rakwireless:nrf52:WisCoreRAK4631Board
Accepted RAK core	1.3.3
Region	plain AS923
Activation	OTAA
Class	A
ADR	enabled
Normal uplink	unconfirmed
Payload	payload-v2 / 46 bytes
DevEUI	ac1f09fffe296d29
JoinEUI	0000000000000000
AppKey	protected local secret; never print/store in docs

Do not substitute AS923-3. Do not copy EMU-01 credentials to SEC.

Files

firmware/EMU01_Agriculture_Node/
EMU01_Agriculture_Node.ino
payload_v2.h
emu01_credentials.example.h
emu01_credentials.h
README.md

The credentials file is local/protected. Never print it in a recorded terminal.

Physical sensor set

RAK1903	OPT3001 light
RAK12010	VEML7700 light
RAK12011	barometer / temperature
RAK1906	BME680 environment
RAK12019	LTR390 UV
RAK12023 + RAK12035	soil moisture / temperature
RAK12005 + RAK12030	rain / water detection

Use the fixed slot map in the sensor preparation manual. Never move modules while powered.

Traffic profiles

Profile	Compile setting	Local sample	Normal uplink	Jitter
Production	default / EMU01_COUNTED_TE ST_PROFILE=0	60 s	300 s	±15 s
Counted test	EMU01_COUNTED_TE ST_PROFILE=1	15 s	15 s	0 s

The counted profile changes timing only. Identity, payload, sensors, decoder, radio region and application path remain unchanged.

Step 1 — Identify the physical board

Before flashing, require:

```
[ ] board is physically identified as EMU-01
[ ] RAK4631 is on the EMU-01 RAK19001 base
[ ] LoRa antenna attached
[ ] seven commissioned sensor types installed
[ ] USB cable supports data
[ ] no recorder/Serial Monitor currently owns the port
```

If board identity is uncertain, stop. Do not flash.

Step 2 — Discover the current serial port

```
Get-CimInstance Win32_SerialPort | Select-Object DeviceID,Description,PNPDeviceID
```

Historical COM numbers are not permanent. DFU can re-enumerate on another port.

Step 3 — Verify Arduino CLI and RAK core

Arduino CLI 1.5.1 was accepted on the workstation but is not guaranteed to be on PATH.

```
Get-ChildItem "$env:LOCALAPPDATA","$env:USERPROFILE" -Filter arduino-cli.exe -Recurse -ErrorAction
SilentlyContinue | Select-Object -First 10 FullName
```

Run the discovered executable:

```
& '<FULL-PATH-TO-arduino-cli.exe>' version
& '<FULL-PATH-TO-arduino-cli.exe>' core list
```

PASS means: Arduino CLI runs and the RAKwireless nRF52 core is present.

Do not update the BSP during a counted experiment series.

Step 4 — Verify required libraries

```
& '<FULL-PATH-TO-arduino-cli.exe>' lib list
```

Accepted dependencies include:

```
SX126x-Arduino  
ClosedCube_OPT3001  
Light_VEML7700  
Adafruit LPS2X  
Adafruit Unified Sensor  
Adafruit BME680  
RAK12019_LTR390  
RAK12035_SoilMoisture
```

If compilation reports LoRaWan-RAK4630.h missing, repair SX126x-Arduino first. That is not an OTAA or ChirpStack failure.

Step 5 — Verify credentials exist without exposing them

```
$P = '.\lorawan-network-server-gateway\firmware\EMU01_Agriculture_Node'; Test-Path "$P\emu01_credentials.h";  
Get-Item "$P\emu01_credentials.h" | Select-Object FullName,Length,LastWriteTime
```

PASS means: the file exists and is non-empty.

Never use Get-Content on the credentials file in a recorded terminal.

Step 6 — Hash the source before building

```
Get-FileHash .\lorawan-network-server-gateway\firmware\EMU01_Agriculture_Node\EMU01_Agriculture_Node.ino -  
Algorithm SHA256  
Get-FileHash .\lorawan-network-server-gateway\firmware\EMU01_Agriculture_Node\payload_v2.h -Algorithm SHA256
```

For counted tests, compare against the accepted build record under chapter4-results/_configuration/emu01-counted-test-15s/.

Do not rebuild the counted artifact during a formal run if the accepted archived artifact already hashes correctly.

Step 7 — Compile

Select:

```
WisBlock Core RAK4631 Board  
rakwireless:nrf52:WisCoreRAK4631Board
```

Production uses the default timing profile. Counted tests use EMU01_COUNTED_TEST_PROFILE=1 only through the accepted build procedure.

A compile PASS proves only source/toolchain compatibility. It does not prove upload, sensors, OTAA or RF.

Step 8 — Upload

Before upload:

```
[ ] antenna attached
[ ] correct physical EMU-01
[ ] Serial Monitor closed
[ ] recorder serial capture paused if a run is active
[ ] current application/DFU COM port selected
```

Successful DFU must include:

```
Device programmed.
```

If the application port disappears and a DFU port appears, rediscover the port instead of retrying the old COM number.

Step 9 — Verify boot profile

Open the current application port at 115200 baud.

Production must show approximately:

```
sample interval = 60000 ms
normal interval = 300000 ms
jitter = 15000 ms
```

Counted testing must show:

```
EMU01_TRAFFIC_PROFILE=COUNTED_TEST_15S
sample interval = 15000 ms
normal interval = 15000 ms
jitter = 0
```

Wrong profile = STOP.

Step 10 — Verify sensors before LoRaWAN

Healthy full cycles should show:

```
valid=0x007F
```

If validity is not 0x007F, troubleshoot sensor power/I2C/module seating before touching ChirpStack.

Step 11 — Verify source transmission lines

A machine-readable SENSOR_TX line should contain sequence, uptime, sensor values, validity, join state and send status.

Healthy joined transmission:

```
join=1
```

```
send_status=0
```

The source sequence is the research recorder's source-attempt authority.

Step 12 — Verify OTAA

Required order:

```
JoinRequest
-> RAK5146 RF reception
-> ChirpStack credential validation
-> JoinAccept
-> EMU-01 reports joined
-> application uplink
```

If OTAA fails, compare protected DevEUI, JoinEUI, AppKey and AS923 configuration. Never print the AppKey while comparing.

Step 13 — Verify the real end-to-end path

```
python .\lorawan-network-server-gateway\test\automation\research-recorder\research_recorder.py preflight --
require-emu --require-sec
```

For EMU-01 acceptance, require at least two consecutive successful SENSOR_TX attempts plus matching Gateway-01/ChirpStack evidence.

Step 14 — Restore production after counted tests

After the counted series:

1. stop/finalize the recorder;
2. restore/build the default production profile;
3. upload it;
4. require the production boot profile;
5. verify OTAA/session and one normal-path uplink;
6. record the restoration.

Do not leave the 15-second counted profile running indefinitely.

Payload-v2 hard rule

Payload-v2 is exactly 46 bytes. Any incompatible field order/size change requires a new payload version plus decoder, Node-RED/application and research-correlation updates.

The USB-only battery sentinel remains battery_mv=0 until a validated real battery voltage source exists.

Troubleshooting order

Symptom	First check
missing header/library	Arduino library/BSP
compile passes, upload fails	COM/DFU/bootloader ownership
Device programmed, no serial	application port / baud / reset
validity not 0x007F	sensor power/I2C/seating
sensors healthy, OTAA fails	DevEUI/JoinEUI/AppKey/AS923
OTAA succeeds, no uplink	send status / antenna / RF
gateway receives, no application row	ChirpStack/application layer
wrong cadence	wrong firmware profile
COM busy	Serial Monitor/orphan recorder child
COM changed after reset	rediscover application/DFU port

Recovery rules

- never erase credentials as a generic fix;
- never change region to force a join;
- never disable LoRaWAN security;
- never rebuild counted firmware mid-trial;
- preserve failed-run evidence;
- hash accepted archived images before reuse.

Completion checklist

- correct EMU-01 identified;
- antenna attached;
- toolchain/core/libraries verified;
- protected credentials present but not exposed;
- correct profile boot behavior;
- healthy validity bitmap;
- OTAA joined;
- two successful SENSOR_TX attempts;
- matching AS923 gateway reception;
- matching ChirpStack/application evidence;
- production restored after counted testing.

Detailed references

- [Firmware README](#)

- [Sensor configuration and LoRaWAN bring-up](#)
- [First-time Arduino walkthrough](#)
- [Research execution baseline](#)

Supporting runbook: README.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

EMU-01 Agriculture Node Firmware

Tracked final-firmware candidate for **EMU-01 / RAK19001 + RAK4631 Core A**. It preserves the frozen 46-byte payload-v2, plain AS923, OTAA, Class A, unconfirmed uplinks, and production scheduler: 60-second local sampling, DevEUI-staggered nominal 5-minute unconfirmed uplinks with ± 15 -second jitter, plus randomized/rate-limited rain-transition event uplinks.

Before compile: select WisBlock Core RAK4631 Board; install the already-proven libraries (SX126x-Arduino, ClosedCube_OPT3001, Light_VEML7700, Adafruit LPS2X, Adafruit Unified Sensor, Adafruit BME680, RAK12019_LTR390, RAK12035_SoilMoisture); copy emu01_credentials.example.h to ignored emu01_credentials.h and fill EMU-01's own values locally. Never commit the AppKey.

Healthy hardware acceptance requires: compile/upload, ten cycles with `valid=0x007F`, OTAA, ten Serial-vs-ChirpStack payload comparisons, then sensor preflight. `battery_mv=0` is the frozen USB-only sentinel until a real battery-mV source is validated.

Reproducible traffic-profile selector

The sketch defines `EMU01_COUNTED_TEST_PROFILE` with default value `0`. An ordinary build therefore remains the accepted production profile. For Chapter IV counted runs only, compile the same source with `EMU01_COUNTED_TEST_PROFILE=1`; this changes timing only and keeps payload-v2, sensors, AS923, OTAA/Class A, credentials, decoder and application path unchanged.

Profile	Definition	Local sample	Normal uplink	Jitter
Production	default / <code>=0</code>	60 s	300 s	± 15 s
Counted test	<code>=1</code>	15 s	15 s	0 s

The boot log is authoritative. Counted-test evidence must contain `EMU01_TRAFFIC_PROFILE=COUNTED_TEST_15S`, sample/normal interval `15000`, and jitter `0`. Production restoration must contain `EMU01_TRAFFIC_PROFILE=PRODUCTION_5MIN`, sample interval `60000`, normal interval `300000`, and jitter `15000`. Archive the exact build settings and compiled artifact SHA-256 with Chapter IV evidence.

Supporting runbook: 01-configure-rak4631-emulators.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

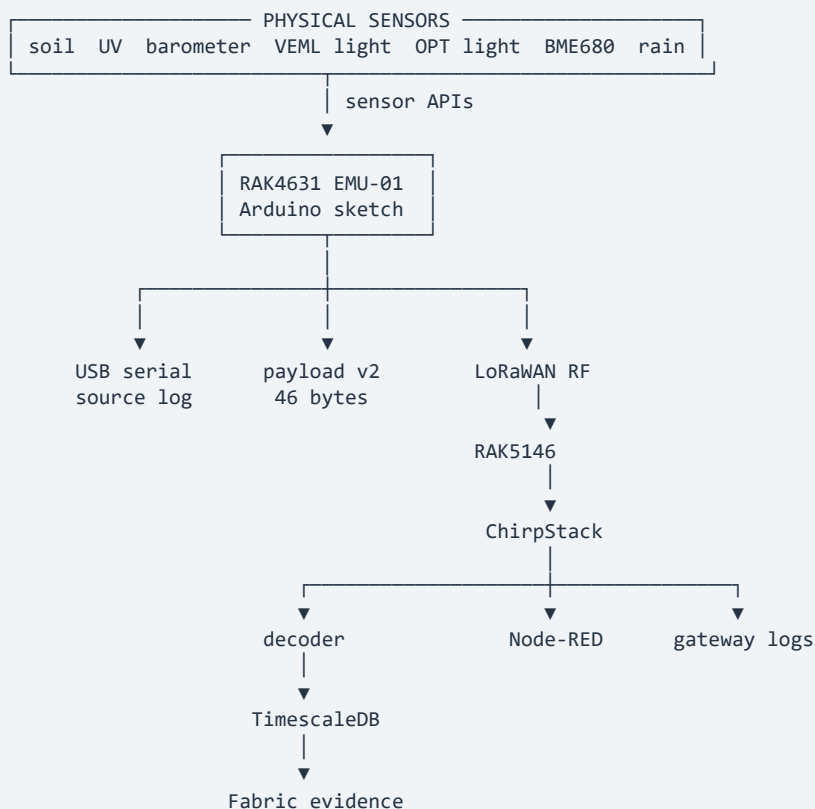
Sensor 1 - Program the RAK4631, Configure LoRaWAN, and Prove the Complete Sensor Path

The filename is retained so existing links do not break. EMU-01 is a real physical Agriculture Kit sensor node. SEC-02 is the separate sensor-verification/security node.

Use this manual **after** all hardware and individual-sensor checks in [assembly/04-verify-all-sensors.md](#) pass.

Architecture

Final EMU-01 path



Firmware lifecycle of the two cores

```
CORE A / EMU-01
Arduino RAK4631
├── individual sensor tests
├── integrated sensor bring-up
└── FINAL full-sensor LoRaWAN firmware

CORE B / SEC-02
Arduino RAK4631 on RAK19007
├── Profile A: A=RAK1903-B, B=RAK12010-B,
│           C=RAK12019-B, D=RAK12011-B,
│           IO=RAK12023+RAK12035-B
├── POWER COMPLETELY OFF
├── Profile B: A=RAK1906-B, B/C/D=EMPTY,
│           IO=RAK12005+RAK12030-B
├── save B-copy evidence
├── remove temporary Sensor/IO modules
├── only AFTER sensor evidence is complete
│   ▼
│   stripped security baseline
│   RAK19007 + Core B + LoRa antenna
│   Sensor A-D = EMPTY; IO = EMPTY
│   ▼
│   security-node firmware path
│   RUI3 conversion if using AT/P2P fixtures
```

Do not convert SEC-02 to RUI3 before its B-copy sensor verification is complete unless you intentionally plan to convert it back to the Arduino firmware family for those tests.

Part A - Final EMU-01 firmware

Step 1 - Confirm the prerequisites

Do not continue unless:

```
[ ] EMU-01 = RAK19001 + Core A
[ ] all seven A-set sensor types installed
[ ] fixed RAK19001 map is A=RAK1903, B=RAK12010, C=RAK12019, D=RAK12011, E=RAK1906, F=EMPTY/RESERVE,
WisIO1=RAK12023, WisIO2=RAK12005
[ ] Pin Mapper result for that exact map is saved with no unresolved conflict
[ ] LoRa antenna attached
[ ] Arduino IDE installed
[ ] RAKwireless Arduino BSP installed
[ ] basic upload to Core A already proven
[ ] every A-copy sensor worked individually
[ ] ten-cycle integrated sensor bring-up already worked
[ ] every B-copy sensor has been or will be verified before SEC-02 security conversion
```

If a sensor cannot initialize in the integrated bring-up sketch, fix that before adding LoRaWAN logic.

Step 2 - Record and freeze the development environment

Create/update:

```
chapter4-results/_device-baseline/EMU-01-firmware.txt
```

Record:

```
Arduino IDE version
RAKwireless Arduino BSP version
selected board = WisBlock Core RAK4631 Board
sensor library versions
LoRaWAN/SX126x library version used by the selected RAK example
final source revision/hash
build date
```

Do not update libraries during a counted experiment group.

Step 3 - Open the final firmware project

Use one dedicated Arduino sketch/project for EMU-01.

Recommended logical layout:

```
EMU01_Agriculture_Node/
├──
├── EMU01_Agriculture_Node.ino
├── sensor_readers.h          # optional helper file
├── payload_v2.h             # optional helper file
└── secrets.h                 # LOCAL ONLY - never commit
```

secrets.h is only an example local organization. If another secure local provisioning mechanism is used, document it. The important rule is that the legitimate AppKey must not enter Git or result/evidence folders.

Step 4 - Select the board and USB port

In Arduino IDE:

1. select WisBlock Core RAK4631 Board;
2. connect only EMU-01 by USB-C data cable;
3. open Tools -> Port;
4. select the port that appears for Core A;
5. confirm the physical board label says EMU-01 before uploading.

Stop if: you are not certain which physical core the selected port belongs to.

Step 5 - Start from the known-good integrated sensor sketch

Do not begin by writing LoRaWAN code into an untested blank sketch.

Use the integrated bring-up sketch accepted in the previous manual as the base:

```
setup()  
├─ serial initialization  
├─ soil init  
├─ UV init  
├─ barometer init  
├─ VEML7700 init  
├─ OPT3001 init  
├─ BME680 init  
└─ rain init
```

Compile once before adding the LoRaWAN portion.

Expected result: the exact full-sensor sketch that already produced ten good sample cycles still builds.

Step 6 - Add the LoRaWAN code using a RAK4631 example as the reference

Use the LoRaWAN example/library supplied or referenced by the pinned RAK4631 BSP instead of writing the radio stack from scratch.

Before compiling LoRaWAN_OTAA_ABP, install the LoRaWAN dependency that provides LoRaWan-RAK4630.h:

```
Arduino IDE  
-> Sketch  
    -> Include Library  
        -> Manage Libraries...
```

Search for:

```
SX126x-Arduino
```

Install the SX126x-Arduino library, then close and reopen the sketch if Arduino IDE does not immediately refresh its include index. The RAK4631-specific header is supplied by this library. If compilation reports:

```
fatal error: LoRaWan-RAK4630.h: No such file or directory
```

the LoRaWAN library is missing or Arduino IDE is not seeing the installed copy; this is not a ChirpStack, OTAA-key, sensor, or radio-region failure.

The final application must provide:

```
OTAA  
Class A  
approved project AS923 configuration  
DevEUI  
JoinEUI/AppEUI  
AppKey  
60-second local sampling schedule plus 5-minute nominal production uplink schedule with fleet  
staggering/jitter  
unconfirmed/confirmed uplink choice frozen before testing
```

Use the LoRaWAN version actually implemented by the pinned build. Do not claim a different MAC version in ChirpStack or the dissertation.

Step 7 - Keep credentials out of the repository

Prepare locally:

```
LEGIT_DEV_EUI
LEGIT_JOIN_EUI
LEGIT_APP_KEY
```

Rules:

```
DevEUI/JoinEUI may be recorded as identifiers
AppKey must not appear in Git
AppKey must not appear in screenshots/results
EMU-01 AppKey must never be copied to SEC-02
EMU-01 session keys must never be copied to SEC-02
SEC-02 may temporarily have its OWN legitimate OTAA AppKey for its separate legitimate-node test
```

The temporary SEC-02 legitimate-node test must use a different DevEUI and different AppKey from EMU-01. After that test is recorded, its temporary legitimate credentials are retired before SEC-02 is converted into the security fixture.

If the AppKey is compiled into the sketch, keep its source definition in an ignored/local secrets file and archive only a **redacted** build record.

Part B - Payload v2

Step 8 - Freeze the binary layout

Final application payload:

Byte(s)	Field	Encoding
0	payload_version	uint8, fixed 2
1-4	test_sequence	uint32, big-endian
5-8	sensor_uptime_ms	uint32, big-endian
9-10	soil_moisture_percent_x100	uint16, big-endian
11-12	soil_temperature_c_x100	int16, big-endian
13-14	uv_index_x100	uint16, big-endian
15-18	barometer_pressure_pa	uint32, big-endian
19-20	barometer_temperature_c_x100	int16, big-endian
21-24	light_veml7700_lux_x100	uint32, big-endian
25-28	light_opt3001_lux_x100	uint32, big-endian
29-30	environment_temperature_c_x100	int16, big-endian
31-32	environment_humidity_percent_x100	uint16, big-endian
33-36	environment_pressure_pa	uint32, big-endian
37-40	environment_gas_resistance_ohm	uint32, big-endian
41	rain_wet	uint8, 0=dry, 1=wet
42-43	battery_mv	uint16, big-endian
44-45	sensor_validity_bitmap	uint16, big-endian

Total = **46 bytes**.

Do not silently change field order or size. Any incompatible layout change requires a new payload version and matching decoder.

UV field check

Before freezing payload v2, prove that the selected RAK12019 driver exposes the quantity you are calling `uv_index`. If it exposes only a different/raw UV quantity, use the correct documented field name and update both firmware and decoder before counted testing rather than mislabeling the value.

Battery field under USB-only testing

Do not fabricate a battery voltage when no battery measurement exists. If the final firmware cannot obtain a meaningful battery value during USB-only operation, use one documented sentinel consistently (for example 0) and state that rule in the payload baseline.

Step 9 - Implement the validity bitmap

```
bit 0 = soil valid
bit 1 = UV valid
bit 2 = barometer valid
bit 3 = VEML7700 valid
bit 4 = OPT3001 valid
bit 5 = RAK1906 environment valid
bit 6 = rain valid
bits 7-15 = 0
```

Healthy full cycle:

```
0b0000000001111111 = 0x007F
```

If a read fails:

```
read fails
├─ clear that sensor validity bit
├─ do NOT silently mark a stale previous sample as valid
└─ still keep this cycle's test_sequence
```

This prevents a sensor failure from being mistaken for an RF packet loss.

Step 10 - Implement one deterministic sample cycle

Use this exact logical order:

```
60-second local sample boundary
┆
┆   ┆
┆   ┆ refresh latest physical sample
┆   ┆
┆   ┆   ┆
┆   ┆   ┆ 5-minute nominal uplink deadline (DevEUI-staggered, ±15 s jitter)
┆   ┆   ┆
┆   ┆   ┆   ┆
┆   ┆   ┆   ┆ increment test_sequence
┆   ┆   ┆   ┆ ┆
┆   ┆   ┆   ┆ ┆   ┆ capture monotonic uptime
┆   ┆   ┆   ┆ ┆   ┆ clear validity bitmap
┆   ┆   ┆   ┆ ┆   ┆
┆   ┆   ┆   ┆ ┆   ┆ read SOIL -> set bit 0 on success
```

```
- read UV    -> set bit 1 on success
- read BARO -> set bit 2 on success
- read VEML -> set bit 3 on success
- read OPT  -> set bit 4 on success
- read ENV  -> set bit 5 on success
- read RAIN -> set bit 6 on success

- pack 46-byte payload
- print one serial source line
- request one LoRaWAN uplink
```

Do not increment `test_sequence` once per sensor. Increment it once per scheduled telemetry cycle.

Step 11 - Use the production fleet scheduler

The final EMU-01 firmware separates local sensing from RF transmission:

```
local physical sampling = every 60 seconds
normal uplink           = nominally every 300 seconds
initial uplink phase    = DevEUI-derived offset within the 5-minute window
per-cycle jitter        = ±15 seconds
normal uplink type      = unconfirmed
ADR                     = enabled
rain event poll         = every 1 second
rain event backoff      = random 0-5 seconds
rain event rate limit   = at most one event uplink per 60 seconds
```

The DevEUI-derived initial phase prevents a fleet that boots together from immediately transmitting together. Per-cycle jitter prevents nodes that drift together from remaining synchronized indefinitely. The implementation uses wrap-safe monotonic `millis()` deadlines instead of `delay(300000)`.

A rain-state transition may schedule one event uplink after randomized backoff. If a normal telemetry deadline is reached at essentially the same time, the normal uplink wins and the pending event is cleared so the node does not immediately transmit duplicate information.

Step 12 - Print one source line per cycle

Use a stable machine-readable format. Example:

```
SENSOR_TX,seq=125,uptime_ms=1875000,soil_pct=41.23,soil_temp_c=27.10,uv_index=0.18,baro_pa=100812,baro_temp_c=27.22,light_veml_lux=382.10,light_opt_lux=401.25,env_temp_c=27.44,env_humidity_pct=68.31,env_pressure_pa=100790,env_gas_ohm=92314,rain_wet=0,battery_mv=0,valid=0x007F,join=1,send_started=1,send_status=0
```

Use the actual measured values. The example numbers are formatting examples only.

The serial source log is authoritative for:

```
test_sequence
physical values sampled for that sequence
validity bitmap
scheduled transmission attempt
reset/rejoin evidence
```

Part C - Build and flash EMU-01

Step 13 - Compile before upload

In Arduino IDE:

1. selected board = RAK4631;
2. selected port = EMU-01;
3. verify required libraries are installed;
4. compile/verify the sketch;
5. resolve all build errors;
6. save the exact accepted source revision/hash.

Do not update random libraries just to make one compiler error disappear without recording the change.

Step 14 - Upload the final firmware

1. Confirm the LoRa antenna is connected.
2. Confirm EMU-01 is physically connected by USB.
3. Confirm you selected EMU-01's port.
4. Click Upload.
5. Wait for successful completion.
6. If the port cannot enter bootloader mode, close Serial Monitor, double-click reset, re-select the bootloader/new port if needed, and retry.
7. Reboot/reset EMU-01 after the successful upload.

Pass: the final sketch starts and prints its startup/sensor status to serial.

Step 15 - Verify local sensor behavior before joining LoRaWAN

Watch serial output and require:

```
all expected sensor initializations reported
RAK1906 stabilization requirement satisfied
soil calibration loaded/applied
both light values present separately
rain state readable
validity = 0x007F during healthy cycles
sequence increases once per actual LoRaWAN transmission attempt; local 60-second samples do not consume
sequence numbers
```

Capture at least ten final-firmware cycles over USB serial before troubleshooting the network.

If the validity bitmap is not 0x007F, fix the sensor first.

Part D - ChirpStack registration and OTAA

Current lab handoff - 2026-08-19

SEC-02 Profile A functional verification	= PASS (operator-confirmed)
SEC-02 Profile B functional verification	= PASS (operator-confirmed)
all seven B-copy sensor types	= FUNCTIONALLY VERIFIED
SEC-02 temporary legitimate DevEUI	= AC1F09FFFE296AEB
SEC-02 legitimate OTAA / ChirpStack connection	= PASS (operator-confirmed)
SEC-02 repeated 15-second application uplinks	= PASS (operator-confirmed)
SEC-02 RAK12011 radio telemetry path	= PASS to ChirpStack LoRaWAN-frame level
SEC-02 root AppKey	= configured locally; NOT recorded here
normal final legitimate ChirpStack node	= EMU-01
next SEC-02 action	= compare decoded RAK12011 values with Serial for several consecutive frames, then retire temporary legitimate mode

SEC-02 is now proven to complete OTAA and continue sending repeated real RAK12011 application uplinks through the RAK5146 gateway into ChirpStack. The accepted implementation sends the first sensor frame shortly after join and then continues on the 15-second application schedule from the normal Arduino loop() context. Keep SEC-02 on the Arduino firmware family only until the decoded temperature/pressure comparison is complete.

Never reuse EMU-01's AppKey or session keys. After the legitimate-node check fully passes, retire the temporary SEC-02 key/device activation before converting it to the security fixture.

Step 15A - Prove a minimal OTAA join before merging the full sensor payload

Do this once before the final 46-byte sensor firmware. It separates LoRaWAN/ChirpStack faults from sensor-integration faults.

On EMU-01:

```
LoRa antenna attached
↓
Arduino IDE
↓
File -> Examples -> LoRaWAN_OTAA_ABP
↓
configure AS923 + OTAA + Class A + credentials
↓
upload the stock Hello! payload
↓
prove JoinRequest -> JoinAccept -> first uplink in ChirpStack
↓
only then merge the seven-sensor payload v2 code
```

For the RAK LoRaWAN_OTAA_ABP example used by the standard RAK4631 Arduino BSP, configure the application values as follows:

```
LoRaMacRegion_t g_CurrentRegion = LORAMAC_REGION_AS923;
bool doOTAA = true;
lmh_confirm g_CurrentConfirm = LMH_UNCONFIRMED_MSG;
DeviceClass_t g_CurrentClass = CLASS_A;
#define LORAWAN_APP_INTERVAL 15000
```

Use unconfirmed uplinks for normal production telemetry. The final EMU-01 firmware uses a nominal 5-minute uplink interval with DevEUI-derived initial staggering and ± 15 -second jitter; the old 15-second cadence is retained

only in historical/bench procedures where explicitly stated. Do not use the default EU868 region.

Frozen lab region rule: the already-configured gateway and ChirpStack server use plain AS923 / region ID and MQTT prefix as923. EMU-01 must therefore use exactly LORAMAC_REGION_AS923. Do **not** use LORAMAC_REGION_AS923_3 for this lab unless the gateway and server are deliberately migrated at the same time.

The RAK example uses these OTAA arrays:

```
uint8_t nodeDeviceEUI[8] = { /* EMU-01 DevEUI, MSB order */ };
uint8_t nodeAppEUI[8]    = { /* EMU-01 JoinEUI/AppEUI, MSB order */ };
uint8_t nodeAppKey[16]   = { /* legitimate AppKey, LOCAL SECRET */ };
```

Do not paste real keys into this Markdown file. Keep the AppKey in a local ignored `secrets.h` or another approved local provisioning path.

ChirpStack 4.x click path for the minimal join

Before creating the device, verify the registered RAK5146 gateway shows a recent `Last seen`. A device cannot join through a gateway path that is not already healthy.

Create the device profile:

```
Tenant
-> Device Profiles
   -> Add device profile
```

Use:

```
Name: EMU-01 RAK4631 AS923
Region: AS923
Region configuration (if shown): as923 / plain AS923
Activation: OTAA
Device class: Class A
```

For the RAK Arduino example based on SX126x-Arduino, the library documents LoRaWAN MAC 1.0.2 and Regional Parameters 1.0.2 Rev B. Use those values when the ChirpStack form exposes them. If the pinned firmware/library is changed later, re-check and match the profile to the actual implementation rather than keeping these values by habit.

For the first join test, leave the payload codec disabled or use a raw/no-op codec. The stock RAK example sends `Hello!`; the Agriculture Kit 46-byte decoder is installed only after the join path is proven.

Create the application if it does not already exist:

```
Tenant
-> Applications
   -> Add application

Name: dissertation-sensors
```

Then create EMU-01:

```
Applications
-> dissertation-sensors
   -> Devices
      -> Add device
```

Use:

```
Name: dissertation-emu-01
DevEUI: <LEGIT_DEV_EUI>
Device profile: EMU-01 RAK4631 AS923
```

In ChirpStack v4, the normal local-join-server flow does not necessarily expose a separate JoinEUI field on the device form. The JoinEUI/AppEUI still exists in the OTAA JoinRequest and must match the value frozen in the EMU-01 firmware. After the device record is created, enter the legitimate root AppKey only on the protected OTAA keys page. Do not place it in screenshots or evidence files.

Minimal OTAA PASS gate

Open both of these ChirpStack views before resetting EMU-01:

```
EMU-01 -> LoRaWAN frames
EMU-01 -> Device data
```

Reset/power-cycle EMU-01. Require this order:

```
JoinRequest
↓
JoinAccept
↓
Join / activation event
↓
UnconfirmedDataUp or ConfirmedDataUp
↓
raw Hello! application payload appears
```

If the gateway sees JoinRequest but the EMU-01 device page does not, check the DevEUI. If ChirpStack sees the JoinRequest but rejects it, check AppKey, JoinEUI routing, device profile MAC/Regional Parameters, and AS923 configuration. If the minimal Hello payload works, freeze the LoRaWAN identity/profile and continue with the full sensor payload; do not change the radio settings while integrating sensors.

Step 15B - Temporarily test SEC-02 as its own legitimate sensor node

Use this stage when SEC-02 must first prove that its radio, OTAA implementation, gateway path, ChirpStack registration, and a real sensor payload all work under valid credentials.

Do this **before RUI3/security conversion**. Keep SEC-02 on the already-proven Arduino RAK4631 firmware family.

Identity separation

Use:

```
device name = dissertation-sec-02-legit
DevEUI      = unique SEC-02/Core-B DevEUI; MUST differ from EMU-01
JoinEUI     = the lab JoinEUI used by this local OTAA flow; all-zero is acceptable only if that is the
frozen lab convention
AppKey      = NEW 16-byte SEC-02-only secret
profile     = dedicated `SEC-02 RAK4631 AS923 LEGIT TEST` profile
class       = A
activation  = OTAA
```

Give SEC-02 its **own device profile** with the same LoRaWAN/region settings as the known-good Arduino RAK4631 profile. This keeps the temporary SEC-02 6-byte test codec isolated from EMU-01's 46-byte payload-v2 codec.

Use:

```
Tenant
-> Device Profiles
   -> Add device profile

Name: SEC-02 RAK4631 AS923 LEGIT TEST
Region: AS923
Region configuration: as923 / plain AS923
MAC version: 1.0.2
Regional Parameters: 1.0.2 Rev B
Device class: A
Codec: temporary SEC-02 6-byte barometer decoder
```

Then create a second device under the test application:

```
Applications
-> dissertation-sensors
   -> Devices
      -> Add device

Name: dissertation-sec-02-legit
DevEUI: AC1F09FFFE296AEB
Device profile: SEC-02 RAK4631 AS923 LEGIT TEST
```

Do not create SEC-02 by cloning EMU-01 credentials. Current temporary legitimate-node DevEUI: AC1F09FFFE296AEB.

After the device record is saved, open SEC-02's **Keys** (OTAA) tab and use ChirpStack's key-generation control to generate a new 128-bit AppKey for SEC-02. Save the generated key in ChirpStack, then copy that exact value only into SEC-02's local firmware secret/provisioning file. Do not paste the key into this repository, screenshots, evidence files, or chat. For LoRaWAN 1.0.x, this is the device's OTAA application root key even if an API/internal field is named `nwkKey`.

If a generated SEC-02 AppKey is accidentally disclosed in chat, screenshots, notes, Git, or evidence, treat that key as retired immediately: use ChirpStack to generate/replace it again before the first OTAA join and never record the exposed value in project documentation.

Fast radio-first check

First run the documented `LoRaWAN_OTAA_ABP` minimal uplink with SEC-02's credentials:

```
LORAMAC_REGION_AS923
OTAA = true
Class A
unconfirmed uplink
15-second interval
SEC-02 DevEUI
SEC-02 AppKey
```

Require:

```
JoinRequest seen by RAK5146
-> JoinAccept generated by ChirpStack
-> SEC-02 prints joined
-> at least one application uplink accepted
```

This isolates OTAA/radio configuration before adding a sensor read.

Real-sensor check

After the minimal join works, use the compile-ready **SEC-02 legitimate RAK12011 OTAA sensor sketch** in [assembly/04b-emu01-sec02-code-reference.md](#). Keep the verified RAK12011-B installed in Sensor D for this check.

Accepted scheduler rule: keep RAK12011 I2C reads, Serial printing, payload packing, and `lmh_send()` in the normal Arduino `loop()` execution path. Use a monotonic `millis()` deadline to trigger the first post-join uplink and each later 15-second attempt. Do not move the sensor read and `lmh_send()` into a `TimerEvent_t` callback. During bench troubleshooting, that callback-based design completed OTAA but stalled before normal application uplinks; the loop-based scheduler produced repeated `UnconfirmedDataUp` frames at the intended cadence.

The sketch sends a six-byte payload containing:

```
bytes 0-1 = RAK12011 temperature C x100, signed int16 big-endian
bytes 2-5 = RAK12011 pressure hPa x100, uint32 big-endian
```

Install the temporary decoder from that same code-reference section or inspect the raw bytes. Require multiple consecutive valid uplinks whose decoded temperature/pressure agree with SEC-02 Serial output within the fixed integer scaling.

Record only non-secret evidence:

```
SEC-02 legitimate OTAA = PASS/FAIL
SEC-02 DevEUI
join UTC window
JoinRequest observed = yes/no
JoinAccept observed = yes/no
number of consecutive real-sensor uplinks
sample serial temperature/pressure
sample decoded temperature/pressure
```

Do not save the AppKey.

Exit from temporary legitimate mode

After PASS:

1. stop SEC-02 transmissions;
2. record the legitimate-node result;
3. remove/disable the temporary dissertation-sec-02-legit device record when it is no longer needed, or at minimum rotate/retire its AppKey before security testing;
4. do not reuse its legitimate session state for attack fixtures;
5. power SEC-02 off before changing Sensor/IO hardware;
6. continue to Part H only when ready to strip the sensors and convert SEC-02 into the isolated security fixture.

Step 16 - Register EMU-01 in ChirpStack

Create one normal OTAA device using:

```
device name: dissertation-emu-01
DevEUI: <LEGIT_DEV_EUI>
JoinEUI/AppEUI: <LEGIT_JOIN_EUI>
```

```
activation: OTAA
class: A
region: plain AS923 (firmware `LORAMAC_REGION_AS923`; ChirpStack region `as923`)
LoRaWAN MAC version: match final firmware
target normal uplink interval: 5 minutes nominal, ±15-second jitter; local sample interval: 60 seconds
```

Enter the legitimate AppKey only in the approved ChirpStack credential field and the secure firmware/provisioning path.

Step 17 - Perform the first OTAA join

Observe simultaneously:

```
EMU-01 serial monitor
RAK5146/gateway evidence
ChirpStack device events
```

Expected sequence:

```
EMU-01 starts
  |
  ▼
JoinRequest transmitted
  |
  ▼
RAK5146 receives
  |
  ▼
ChirpStack validates OTAA
  |
  ▼
JoinAccept
  |
  ▼
EMU-01 joined
```

Stop if: sensor reads work but OTAA fails. Troubleshoot antenna, AS923 configuration, EUI/key values, device profile, and gateway reception rather than changing sensor libraries.

Part E - ChirpStack decoder

Step 18 - Install the payload-v2 codec

Use a dedicated codec that rejects incorrect versions/lengths:

```
function decodeUplink(input) {
  const b = input.bytes;
  if (!b || b.length !== 46) {
    return { errors: ["expected 46-byte Agriculture Kit payload v2"] };
  }
  if (b[0] !== 2) {
    return { errors: ["unsupported Agriculture Kit payload version"] };
  }

  const u16 = (i) => (b[i] << 8) | b[i + 1];
  const s16 = (i) => {
```

```

    const v = u16(i);
    return v & 0x8000 ? v - 0x10000 : v;
};
const u32 = (i) =>
  ((b[i] * 0x1000000) +
   (b[i + 1] << 16) +
   (b[i + 2] << 8) +
   b[i + 3]) >>> 0;

const environmentTemperature = s16(29) / 100.0;
const environmentHumidity = u16(31) / 100.0;

return {
  data: {
    payload_version: b[0],
    test_sequence: u32(1),
    sensor_uptime_ms: u32(5),
    soil_moisture_percent: u16(9) / 100.0,
    soil_temperature_c: s16(11) / 100.0,
    uv_index: u16(13) / 100.0,
    barometer_pressure_pa: u32(15),
    barometer_temperature_c: s16(19) / 100.0,
    light_veml7700_lux: u32(21) / 100.0,
    light_opt3001_lux: u32(25) / 100.0,
    environment_temperature_c: environmentTemperature,
    environment_humidity_percent: environmentHumidity,
    environment_pressure_pa: u32(33),
    environment_gas_resistance_ohm: u32(37),
    rain_wet: b[41] === 1,
    battery_v: u16(42) / 1000.0,
    sensor_validity_bitmap: u16(44),

    // Existing convenience fields used by the test data path.
    temperature_c: environmentTemperature,
    humidity_percent: environmentHumidity
  }
};
}

```

If Step 8 changed the UV field semantics/name, change the codec consistently before freezing the payload contract.

Step 19 - Compare serial source to decoded output

For at least ten consecutive sequences:

```

EMU-01 source line
  |
  | test_sequence
  | every physical sensor field
  | validity bitmap
  |
  ▼
ChirpStack decoded object

```

Pass only when the same sequence has the expected values after the defined integer scaling/rounding.

Part F - Node-RED and TimescaleDB

Step 20 - Keep all physical fields

Store the complete decoded object in `payload_json` and normalize at least:

```
soil_moisture_percent      %
soil_temperature_c        Cel
uv_index                   1 (only if truly UV index)
barometer_pressure_pa     Pa
barometer_temperature_c   Cel
light_veml17700_lux       lx
light_opt3001_lux         lx
environment_temperature_c Cel
environment_humidity_percent %
environment_pressure_pa   Pa
environment_gas_resistance_ohm ohm
rain_wet                   boolean/state
battery_v                  V
```

Compatibility fields:

```
telemetry.uplinks.temperature_c = environment_temperature_c
telemetry.uplinks.humidity_percent = environment_humidity_percent
telemetry.uplinks.battery_v = battery_v
```

Use quality measured for valid physical measurements. Do not label the sensor data synthetic.

Step 21 - Prove one complete end-to-end record

Pick one `test_sequence` and prove:

```
EMU-01 serial source
  | same seq/value
  ▼
ChirpStack decoder
  | same seq/value
  ▼
Node-RED
  | same seq/value
  ▼
TimescaleDB payload_json
  |
  ▼
selected Fabric evidence
```

At minimum verify:

```
test_sequence
sensor_validity_bitmap
soil value
UV value
barometer value
both light values
RAK1906 values
rain state
```

If any field differs beyond the documented integer scaling/rounding, stop and fix the codec/mapping before counted testing.

Part G - Freeze EMU-01

Step 22 - Create the final baseline

Save:

```
chapter4-results/_device-baseline/  
  EMU-01-hardware.txt  
  EMU-01-firmware.txt  
  EMU-01-sensor-map.txt  
  EMU-01-payload-contract-v2.txt  
  EMU-01-sensor-library-versions.txt  
  chirpstack-physical-sensor-codec.js  
  firmware-source-or-build-hash.txt  
  as923-radio-baseline.txt
```

Do not save keys.

After the baseline is frozen, do not modify the sensor map, payload structure, firmware, BSP, or sensor libraries inside the same counted repetition group.

Part H - SEC-02 after sensor verification

Step 23 - Confirm every B-copy sensor was already tested

Before changing SEC-02 firmware family, require evidence for:

```
SOIL-B  
UV-B  
BARO-B  
LIGHT-VEML-B  
LIGHT-OPT-B  
ENV-B  
RAIN-B
```

This is the hard boundary between sensor verification and security node.

The accepted physical profiles are frozen in [assembly/02b-rak19007-sec02-fixed-profiles.md](#). Before changing firmware family, also require:

```
[ ] Profile A readings and pin-map evidence saved  
[ ] Profile B readings and pin-map evidence saved  
[ ] USB/battery/solar disconnected before the final rebuild  
[ ] Sensor A = EMPTY  
[ ] Sensor B = EMPTY  
[ ] Sensor C = EMPTY  
[ ] Sensor D = EMPTY  
[ ] IO slot = EMPTY
```

```
[ ] RAK4631 Core B remains installed
[ ] LoRa antenna remains attached
```

Do not leave a temporary Agriculture Kit sensor on SEC-02 merely because it was used during verification. The normal security baseline is intentionally stripped to reduce unrelated hardware variables.

Step 24 - Decide the security firmware path

The current test manuals use RUI3/AT-command fixtures for invalid OTAA and raw LoRa P2P security transmissions.

If Core B is still a standard Arduino-BSP RAK4631, convert it to RAK4631-R/RUI3 **only after B-copy verification** using RAKwireless' current official RAK4631-R DFU/conversion procedure.

Do not copy old bootloader/DFU package names blindly from a screenshot or old note. Use the currently published RAK conversion guide and record the exact package/version used.

Physical conversion acceptance - 2026-09-02: PASS. The second RAK4631 plugged in by the operator as SEC-01 is the hardware role this repository has historically documented as SEC-02; do not interpret those labels as two different security nodes unless the project topology is deliberately revised. The board was positively identified on COM13 as the RAK4631 Arduino USB identity VID_239A/PID_8029. The current official RAK two-stage Windows conversion was then executed. The RUI3 bootloader package SHA-256 was F39A6B14964F1CF308908FD4EBAF068316A4827B1E5EFC08F4BF04EAAE33483B; after the 1200-baud transition the DFU port appeared as COM14, VID_239A/PID_002A, and the retry directly on that DFU port reported Device programmed. The final RUI3 package SHA-256 was 7885110C39FA2AF1B207026546A4F33122B81AC7C825939AE3B1C654D7EF155B; Nordic nrfutil 6.1.7 (executable SHA-256 FEE3AEED1843A4BFFC082747CEA60DC072929EB36E67D62C8A3FD7471E0DC92D) programmed it successfully. The application re-enumerated as COM16, VID_1915/PID_521F.

The initial converted RUI3 identity was 3.4.2-rui3_22q1_update.112. During the 2026-09-03 security preflight that old build was found unable to expose the public-LoRa syncword/IQ controls required for a faithful raw LoRaWAN replay/spoof waveform, so SEC was upgraded in place through the official RAK4631-R USB DFU path to **RUI_4.2.4_RAK4631**. The downloaded RAK4631_latest_dfu_package.zip SHA-256 was 738C317DCCDF19B987471AB0620B5AB3CFF6B920A896BC744D2A785EDA9D55CC; Nordic nrfutil 6.1.7 remained SHA-256 FEE3AEED1843A4BFFC082747CEA60DC072929EB36E67D62C8A3FD7471E0DC92D, and DFU returned Device programmed.

RUI3 4.2.4 exposed AT+SYNWORD and AT+IQINVER. SEC was configured temporarily at 923200000 Hz, SF7, 125 kHz, CR 4/5, preamble 8, TX power 14, public LoRaWAN syncword 3444, and normal uplink IQ. One uncounted short P2P rehearsal returned TXP2P DONE; Gateway-01 then received a distinct SF7/923.2 MHz raw frame with gateway uplink ID 1355632844. That uplink ID had no EMU-01 telemetry row, while the nearest legitimate EMU-01 frame used a different uplink ID, proving the RAK5146 raw-RF rehearsal independently. SEC was immediately returned to the safe parked state: NWM=1, BAND=8 / **AS923-1**, NJM=1 (OTAA), CLASS=A, NJS=0, and DEVEUI=0000000000000000. No EMU-01 credential/session material was installed or reused.

After conversion, verify:

```
AT+VER=?
AT+BUILDTIME=?
AT+REPOINFO=?
AT+HWMODEL=?
AT+HWID=?
```

Save sanitized output as:

Step 25 - Configure the wrong-AppKey fixture

Rules:

```
EMU-01 = powered off for this identity-overlap fixture
SEC-02 may use LEGIT_DEV_EUI + LEGIT_JOIN_EUI
SEC-02 MUST use WRONG_APP_KEY
SEC-02 MUST NOT receive LEGIT_APP_KEY
```

Representative RUI3 sequence:

```
AT+NWM=1
AT+NJM=1
AT+BAND=<APPROVED AS923 BAND VALUE>
AT+DEVEUI=<LEGIT_DEV_EUI>
AT+APPEUI=<LEGIT_JOIN_EUI>
AT+APPKEY=<WRONG_APP_KEY>
AT+JOIN=1:0:10:1
AT+NJS=?
```

Do not save the key value in the evidence transcript.

Step 26 - Configure the unregistered-device fixture

Use:

```
DevEUI = <UNREGISTERED_DEV_EUI>
JoinEUI = dedicated test JoinEUI
AppKey = dedicated test fixture key
```

Confirm the test DevEUI does **not** exist in ChirpStack before counting an unregistered-device attempt.

Step 27 - Prepare raw LoRa P2P mode

Representative RUI3 controls:

```
AT+NWM=0
AT+P2P=<freq>:<sf>:<bw>:<cr>:<preamble>:<power>
AT+SYNCWORD=<captured/required LoRaWAN-compatible value>
AT+IQINVER=<uplink-compatible value>
AT+PCRYPT=0
AT+PRECV=0
AT+PSEND=<RAW_PHYPAYLOAD_HEX>
```

Do not guess RF settings. Copy frequency/data-rate-related parameters from the actual legitimate captured packet/gateway evidence used by the authorized test.

Step 28 - Prove SEC-02 RF reception before counted replay/spoof tests

```
SEC-02 raw transmit
  |
  v
RAK5146 receives RF
  |
  v
gateway evidence saved
```

If RAK5146 does not receive the rehearsal frame, replay/spoofing tests are blocked even if EMU-01 LoRaWAN works normally.

Final setup acceptance before sensor preflight

Do not enter counted execution from this manual. First satisfy this setup gate, then run the dedicated sensor preflight:

```
[ ] both RAK4631 cores have permanent identities/labels
[ ] all A/B direct sensors were physically verified
[ ] EMU-01 keeps complete A-set installed
[ ] final Arduino firmware uploaded to EMU-01
[ ] validity bitmap = 0x007F during healthy pre-run cycles
[ ] payload v2 frozen at 46 bytes
[ ] production scheduler proven: 60-second local sampling, DevEUI-staggered 5-minute nominal unconfirmed
uplinks with ±15-second jitter, and rate-limited randomized rain-event uplinks
[ ] source log matches ChirpStack decoder for ten sequences
[ ] EMU-01 OTAA join succeeds
[ ] one physical-sensor record reaches TimescaleDB
[ ] one selected record completes the Fabric evidence path
[ ] firmware/BSP/library versions are recorded
[ ] no legitimate keys are present in Git/evidence/SEC-02
[ ] SEC-02 B-copy verification completed before security conversion
[ ] SEC-02 wrong-AppKey fixture works
[ ] SEC-02 unregistered-device fixture works
[ ] SEC-02 raw LoRa frame is received by RAK5146
```

When this setup gate passes, continue to [preflight/00-README.md](#). The preflight re-tests the **frozen final configuration as one system** and must produce `SENSOR_PREFLIGHT_STATUS=GO` before Execution 01 begins.

Official references

- RAK4631 quick start: <https://docs.rakwireless.com/product-categories/wisblock/rak4631/quickstart/>
- RAK4631 overview: <https://docs.rakwireless.com/product-categories/wisblock/rak4631/overview/>
- WisBlock quick start: <https://docs.rakwireless.com/product-categories/wisblock/quickstart/>
- RAK4631-R conversion / DFU: <https://docs.rakwireless.com/product-categories/wisblock/rak4631-r/dfu/>
- RUI3 AT commands: <https://docs.rakwireless.com/product-categories/software-apis-and-libraries/rui3/at-command-manual/>
- Agriculture Kit: <https://docs.rakwireless.com/product-categories/wisblock/kit7-agriculture/overview/>

Supporting runbook: 04a-first-time-arduino-operator-walkthrough.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Sensor Assembly 4A - First-Time Arduino IDE Walkthrough for EMU-01

Use this manual if you are new to Arduino IDE. It turns the short software steps in `04-verify-all-sensors.md` into a click-by-click procedure and explains **why** each step exists.

For actual copy/paste code, use [04b-emu01-sec02-code-reference.md](#) as the authoritative source for both EMU-01 and SEC-02. This walkthrough may repeat code for teaching context, but a chat-only or undocumented sketch must not become the accepted lab baseline.

This is the recommended first-time path after EMU-01 has been physically assembled and has passed the pre-power inspection.

Do **not** start with ChirpStack or LoRaWAN. First prove the local chain one layer at a time:

```
physical assembly
  ↓
USB device appears
  ↓
Arduino IDE + RAK BSP
  ↓
sketch compiles
  ↓
RAK4631 accepts firmware
  ↓
RAK4631 executes firmware
  ↓
USB Serial works
  ↓
first real sensor works
  ↓
second real sensor works
  ↓
remaining sensors
  ↓
integrated sensor firmware
  ↓
LoRaWAN / ChirpStack
```

Why this order matters:

If seven sensors + LoRaWAN are added immediately and something fails, we do not know whether the fault is:

```
Arduino setup?
USB cable?
bootloader?
RAK4631?
sensor library?
I2C?
```

wrong slot?
bad sensor?
LoRaWAN?

Testing one layer at a time makes the first failing layer obvious.

Part 1 - Confirm EMU-01 before USB power

The permanent EMU-01 map must be:

```
RAK19001 / EMU-01

Sensor A = RAK1903   OPT3001 ambient light
Sensor B = RAK12010  VEML7700 ambient light
Sensor C = RAK12019  UV
Sensor D = RAK12011  barometer
Sensor E = RAK1906   BME680 environment
Sensor F = EMPTY / NA

WisIO 1  = RAK12023 -> RAK12035 soil probe
WisIO 2  = RAK12005 -> RAK12030 rain pad

CPU      = RAK4631
```

Before connecting USB:

```
[ ] RAK4631 fully seated in CPU slot
[ ] retaining screws installed
[ ] no loose screw or conductive debris
[ ] LoRa antenna attached to the LoRa RF connector
[ ] Sensor B contains RAK12010 / VEML7700
[ ] Sensor F is empty / reserve
[ ] all other modules match the fixed map
[ ] soil/rain electronics are dry
[ ] optical sensors are unobstructed
[ ] BME680 has airflow
```

Why attach the LoRa antenna now? Later test sketches or examples can enable the radio. Keeping the correct antenna attached avoids accidentally operating the radio without its intended load.

Stop if: any module is in the wrong slot, a connector is partly seated, or anything becomes abnormally hot after power is applied.

Part 2 - First USB connection

Use USB-C only for initial bring-up:

```
Laptop
|
USB-C DATA cable
|
▼
RAK19001 + RAK4631
```

Do not add battery or solar during this stage.

Step 2.1 - Connect EMU-01

1. Keep SEC-02 disconnected.
2. Connect a known-good USB-C **data** cable to EMU-01.
3. Wait for Windows to detect the board.
4. Watch for abnormal heat, smell, smoke, unstable LEDs, or repeated USB disconnects.
5. If anything abnormal occurs, unplug immediately.

Why use a data cable? Some USB-C cables provide power only. Arduino programming needs both power and USB data.

Part 3 - Install Arduino IDE and the RAK4631 board support

Step 3.1 - Install Arduino IDE

1. Download Arduino IDE from the official Arduino distribution.
2. On Windows, prefer the normal Arduino installer rather than the Microsoft Store build if third-party BSP installation causes problems.
3. Start Arduino IDE.

Pass: Arduino IDE opens normally.

Step 3.2 - Add the RAKwireless board package URL

In Arduino IDE:

```
File
-> Preferences
    -> Additional Boards Manager URLs
```

Add:

```
https://raw.githubusercontent.com/RAKwireless/RAKwireless-Arduino-BSP-Index/main/package_rakwireless_index.json
```

If another URL is already present, add the RAK URL as another entry rather than deleting the existing one.

Why? Arduino IDE does not know how to compile for a RAK4631 by default. This URL tells Boards Manager where to obtain the RAKwireless board definition, compiler settings, core files, and upload tools.

Step 3.3 - Install the RAKwireless BSP

Open:

```
Tools
-> Board
```

-> Boards Manager

Then:

1. search RAKwireless OR RAK;
2. install the RAKwireless nRF52/WisBlock board package;
3. record the installed version in `arduino-environment.txt`.

The currently observed lab machine used RAKwireless nRF52 BSP 1.3.3. Record the actual installed version; do not silently update it during a counted experiment group.

Why? The BSP supplies the RAK4631-specific Arduino core and DFU/upload tooling.

Step 3.4 - Select the RAK4631 board

Choose:

```
Tools
-> Board
    -> RAKwireless ...
        -> WisBlock Core RAK4631 Board
```

The exact submenu name can vary by BSP version. The selected board name that matters is:

WisBlock Core RAK4631 Board

Part 4 - Identify the correct COM port

With EMU-01 connected:

```
Tools
-> Port
```

On Windows the board appears as a COM port, for example COM12.

If you are unsure which port is EMU-01:

```
disconnect EMU-01
↓
look at Tools -> Port
↓
reconnect EMU-01
↓
look again
↓
new COM port = EMU-01
```

Select that port.

Why? Arduino IDE needs both the correct board definition and the correct physical serial/DFU port. Selecting the right board but the wrong COM port still prevents upload.

Do not permanently hard-code the COM number in the documentation. The port can change when the RAK4631 enters bootloader mode or after reconnecting USB.

Part 5 - First harmless RAK4631 program

Do this before any sensor code.

Step 5.1 - Create a new sketch

Open:

```
File
-> New Sketch
```

Delete the default contents and paste:

```
#include <Adafruit_TinyUSB.h>

void setup()
{
  Serial.begin(115200);
  delay(2000);

  Serial.println();
  Serial.println("=====");
  Serial.println("EMU-01 RAK4631 STARTED");
  Serial.println("USB SERIAL TEST = PASS");
  Serial.println("=====");
}

void loop()
{
  Serial.println("EMU-01 is alive");
  delay(1000);
}
```

Why this sketch is intentionally simple

It tests only:

```
compiler
+
RAK BSP
+
DFU upload
+
RAK4631 CPU execution
+
USB Serial
```

No sensor library and no LoRaWAN code is involved yet.

Why Adafruit_TinyUSB.h is included here

The RAK nRF52 BSP uses TinyUSB for USB functionality. On the observed Windows lab setup with RAKwireless nRF52 BSP 1.3.3, a sketch using Serial without this include produced linker errors such as:

```
undefined reference to `Adafruit_USBD_CDC::begin(unsigned long)'
undefined reference to `Adafruit_USBD_CDC::operator bool()'
undefined reference to `Serial'
```

Adding:

```
#include <Adafruit_TinyUSB.h>
```

resolved that specific build problem.

Treat this as a **known lab/BSP compatibility fix**, not proof that every future RAK BSP release will require an explicit TinyUSB include.

Part 6 - Understand Verify / Compile

At the upper-left of Arduino IDE:



Click the **checkmark / Verify** first.

What Verify actually does

```
Arduino source code
  ↓
compiler checks syntax
  ↓
libraries are compiled
  ↓
linker combines sketch + RAK core + libraries
  ↓
final firmware image is created
```

A successful compile ends with output similar to:

```
Sketch uses 44880 bytes (5%) of program storage space. Maximum is 815104 bytes.
Global variables use 8252 bytes (3%) of dynamic memory, leaving ...
```

The exact numbers change with the sketch.

Meaning: the laptop successfully created firmware for the selected RAK4631 target.

It does **not** yet prove that the physical board was programmed or that the code runs.

If Verify fails with a missing header

Example:

```
fatal error: Light_VEML7700.h: No such file or directory
```

Meaning:

```
Arduino reached #include <...>
  ↓
requested library/header is not installed
  ↓
compilation stops before hardware is touched
```

Do not troubleshoot the sensor hardware for a missing-header error. Install the correct library first.

Part 7 - Upload to the physical RAK4631

After Verify passes, click the **right-arrow Upload** button.

A normal RAK4631 DFU upload may show output similar to:

```
Upgrading target on COMxx with DFU package ...
...
Activating new firmware
Device programmed.
```

Why Device programmed. matters

Compile only proved:

```
laptop can build firmware
```

Device programmed. proves:

```
laptop
  ↓
Arduino IDE
  ↓
RAK DFU / bootloader
  ↓
firmware written into the physical RAK4631
```

It still does not prove the application is executing correctly. Serial Monitor is the next layer.

If upload fails

Try in this order:

1. close Serial Monitor
2. double-click RESET on the WisBlock base
3. open Tools -> Port again
4. select the bootloader/new COM port if it changed
5. click Upload again

Do not start changing sensor wiring because of a DFU/COM-port problem.

Part 8 - Prove the uploaded firmware is actually executing

Open:

```
Tools
-> Serial Monitor
```

Set:

```
115200 baud
```

Expected output:

```
=====
EMU-01 RAK4631 STARTED
USB SERIAL TEST = PASS
=====
EMU-01 is alive
EMU-01 is alive
EMU-01 is alive
...
```

If the startup banner scrolled past before Serial Monitor opened, seeing repeated:

```
EMU-01 is alive
```

is enough to prove the loop is running.

Why this is the first major gate

At this point you have proven:

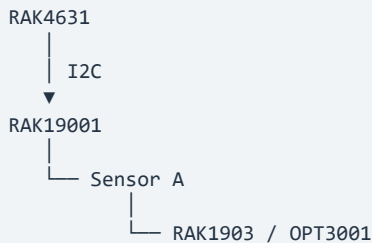
Arduino IDE installed	PASS
RAK BSP installed	PASS
RAK4631 selected	PASS
USB connection	PASS
firmware compilation	PASS
DFU / bootloader upload	PASS
RAK4631 CPU execution	PASS
USB Serial output	PASS

Only now should sensor debugging begin.

Part 9 - First physical sensor: RAK1903 / OPT3001 in Sensor A

Start with RAK1903 because it is a simple I2C sensor and its response is easy to prove physically by changing the light level.

Relevant path:



Step 9.1 - Install the OPT3001 library

Open Library Manager:

```
Sketch
-> Include Library
    -> Manage Libraries...
```

or use the Library Manager icon in the left sidebar.

Search:

```
ClosedCube OPT3001
```

Install the library that provides:

```
#include <ClosedCube_OPT3001.h>
```

Why? The library contains the sensor register/protocol logic. The RAK4631 does not automatically know how to configure an OPT3001 or convert its raw value into lux.

Step 9.2 - Use the RAK1903 test sketch

Create a new sketch and paste:

```
#include <Adafruit_TinyUSB.h>
#include <Wire.h>
#include <ClosedCube_OPT3001.h>

#define OPT3001_ADDRESS 0x44

ClosedCube_OPT3001 lightSensor;

void setup()
{
    // RAK19001 switched 3V3_S sensor power.
```

```

pinMode(WB_I02, OUTPUT);
digitalWrite(WB_I02, HIGH);
delay(500);

Serial.begin(115200);
delay(2000);

Serial.println();
Serial.println("=====");
Serial.println("EMU-01 RAK1903 TEST");
Serial.println("=====");

Wire.begin();
lightSensor.begin(OPT3001_ADDRESS);

Serial.print("Manufacturer ID: 0x");
Serial.println(lightSensor.readManufacturerID(), HEX);

Serial.print("Device ID: 0x");
Serial.println(lightSensor.readDeviceID(), HEX);

OPT3001_Config config;
config.RangeNumber = B1100;
config.ConversionTime = B0;
config.Latch = B1;
config.ModeOfConversionOperation = B11;

OPT3001_ErrorCode error = lightSensor.writeConfig(config);

if (error == NO_ERROR)
{
    Serial.println("RAK1903 configuration = PASS");
}
else
{
    Serial.print("RAK1903 configuration ERROR = ");
    Serial.println(error);
}
}

void loop()
{
    OPT3001 result = lightSensor.readResult();

    if (result.error == NO_ERROR)
    {
        Serial.print("RAK1903 light = ");
        Serial.print(result.lux);
        Serial.println(" lux");
    }
    else
    {
        Serial.print("RAK1903 read ERROR = ");
        Serial.println(result.error);
    }

    delay(1000);
}

```

Why WB_I02 is set HIGH

On the RAK19001, WB_I02 controls the switched 3V3_S sensor power rail used by the project build:

```

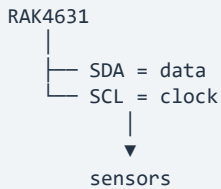
WB_I02 HIGH
↓
3V3_S enabled
↓
sensor modules powered

```

This is one reason the permanent project map leaves Sensor B unused.

Why wire.h is present

Wire is Arduino's I2C layer:



The OPT3001 is addressed at 0x44 in the RAK example.

Step 9.3 - Verify, Upload, Serial Monitor

Do the same three-layer check:

```
Verify ✓
↓
firmware builds
↓
Upload →
↓
Device programmed.
↓
Serial Monitor @ 115200
```

Expected output contains changing lux values, for example:

```
RAK1903 configuration = PASS
RAK1903 light = ... lux
RAK1903 light = ... lux
```

The exact lux value is not the pass criterion.

Step 9.4 - Physical response test

1. leave RAK1903 exposed to normal room light and observe lux;
2. cover the **RAK1903 in Sensor A** with an opaque object or finger;
3. confirm lux drops;
4. uncover it;
5. confirm lux rises again.

Why this matters:

```
number appears
= communication probably works

light changes
↓
sensor value changes
= physical sensor response proven
```

PASS: RAK1903 initializes, returns numeric lux values, and responds correctly to changing illumination.

Part 10 - Second physical sensor: RAK12010 / VEML7700 in Sensor B

Test RAK12010 next because it is another easy I2C device and gives a second independent proof that multiple sensor modules can use the shared bus.

Permanent position:

```
Sensor B = RAK12010 / VEML7700
```

Step 10.1 - Install the correct VEML7700 library

Open Library Manager and search:

```
VEML7700
```

Install the **RAKWireless VEML7700** library used by the RAK12010 example. It must provide:

```
#include "Light_VEML7700.h"
```

Do not assume another VEML7700 library is interchangeable. For example, a library using `Adafruit_VEML7700.h` has a different API and will not satisfy a sketch that asks for `Light_VEML7700.h`.

Exact meaning of this common error

If Verify shows:

```
fatal error: Light_VEML7700.h: No such file or directory
compilation terminated.
```

then:

```
RAK4631 hardware      not tested by this compile
RAK12010 hardware     not tested by this compile
I2C bus               not tested by this compile

problem = required header/library is missing
```

Fix:

```
Library Manager
  ↓
search VEML7700
  ↓
install RAKWireless-compatible library
  ↓
Verify again
```

Do not remove or reseal the physical RAK12010 merely because of a missing-header compiler error.

Step 10.2 - Use the RAK12010 test sketch

Create a new sketch:

```
#include <Adafruit_TinyUSB.h>
#include "Light_VEML7700.h"

Light_VEML7700 lightSensor = Light_VEML7700();

void setup()
{
  // Enable RAK19001 switched sensor power.
  pinMode(WB_IO2, OUTPUT);
  digitalWrite(WB_IO2, HIGH);
  delay(500);

  Serial.begin(115200);
  delay(2000);

  Serial.println();
  Serial.println("=====");
  Serial.println("EMU-01 RAK12010 TEST");
  Serial.println("=====");

  if (!lightSensor.begin())
  {
    Serial.println("RAK12010 NOT FOUND");
    Serial.println("TEST = FAIL");

    while (1)
    {
      delay(1000);
    }
  }

  lightSensor.setGain(VEML7700_GAIN_1);
  lightSensor.setIntegrationTime(VEML7700_IT_800MS);

  Serial.println("RAK12010 detected");
  Serial.println("TEST INITIALIZATION = PASS");
}

void loop()
{
  Serial.print("RAK12010 light = ");
  Serial.print(lightSensor.readLux());
  Serial.println(" lux");
  delay(1000);
}
```

This uses the same `Light_VEML7700` class, `begin()`, gain setting, integration-time setting, and `readLux()` pattern as the current RAK12010 example.

Step 10.3 - Verify, Upload, observe

```
Verify ✓
  ↓
compile passes
  ↓
Upload →
  ↓
Device programmed.
```

↓
Serial Monitor @ 115200

Expected:

```
RAK12010 detected
TEST_INITIALIZATION = PASS
RAK12010 light = ... lux
RAK12010 light = ... lux
```

Step 10.4 - Physical response test

Make sure you manipulate the **RAK12010 in Sensor B**, not the already-tested RAK1903 in Sensor A:

```
normal room light
↓
record lux
↓
cover RAK12010
↓
lux should decrease
↓
uncover RAK12010
↓
lux should increase
```

PASS: RAK12010 initializes, reports numeric lux, and responds correctly to changing illumination.

Step 10.5 - Why RAK12010 is now in Sensor B

The 2026-09-02 layout review moved RAK12010 from lower-side Sensor F to top-side Sensor B.

```
Sensor B = RAK12010 / VEML7700
Sensor F = EMPTY / reserve
```

This is electrically valid because RAK documents RAK12010 for Sensor Slots A through F and its WisConnector uses only I2C, 3V3_S, and GND. It does **not** connect an interrupt line to Slot-B WB_I02; WB_I02 remains the RAK19001 shared 3V3_S enable. The same safe B-slot principle is already used by the SEC-02 Profile A layout.

The change improves measurement quality and assembly simplicity: VEML7700 no longer depends on a lower-side light path or bench standoffs, and Sensor F becomes a reserve slot while WB_I06 stays dedicated to the rain path.

For bench and enclosure testing:

```
[ ] keep the Sensor-B VEML7700 optical surface unobstructed
[ ] keep cables and other PCBs out of its light path
[ ] provide a clear enclosure opening/window appropriate to the intended light measurement
[ ] verify cover -> lux decreases and uncover -> lux increases
```

Moving F -> B does not require a firmware code change because the VEML7700 path is I2C-only. It **does** require a revised Pin Mapper record plus a fresh RAK12010 response check and integrated EMU-01 sensor/preflight verification before the layout is frozen.

Keep the two final fields separate:

```
RAK1903 -> light_opt3001_lux
RAK12010 -> light_vem17700_lux
```

Part 11 - Third physical sensor: RAK12011 barometer in Sensor D

After both light sensors pass, test the RAK12011 next. The RAK12011 uses an ST LPS33HW pressure sensor and reports both barometric pressure and temperature over I2C.

Permanent position:

```
Sensor D = RAK12011 / LPS33HW
```

RAK allows RAK12011 in Sensor A and C-F. Sensor D is therefore valid and, in our fixed map, gives the module the WB_I05 digital-output/interrupt role while avoiding WB_I02, which controls 3V3_S.

Step 11.1 - Why test the barometer now?

We already proved two I2C light sensors. The barometer adds a different sensor family while still using the known-good I2C path:

```
RAK4631 + USB/Serial      already proven
RAK19001 3V3_S            already proven
I2C bus                  already proven
↓
new device: RAK12011
↓
pressure + temperature
```

If this sensor now fails with `Sensor not found`, troubleshooting can focus on the RAK12011 library/address/Slot-D connection instead of re-questioning the entire Arduino setup.

Step 11.2 - Install the required library

Open Arduino IDE Library Manager:

```
Sketch
-> Include Library
    -> Manage Libraries...
```

Search:

```
Adafruit LPS2X
```

Install the **Adafruit LPS2X** library used by RAK's current RAK12011 example. If Arduino asks to install dependencies such as **Adafruit Unified Sensor**, install the required dependencies as well.

The test sketch needs:

```
#include <Adafruit_LPS2X.h>
#include <Adafruit_Sensor.h>
```

If Verify reports No such file or directory for either header, stop and fix the library installation first. That is a compile-time software problem; the physical RAK12011 has not been tested yet.

Step 11.3 - Create the RAK12011 test sketch

Create a new sketch and paste:

```
#include <Adafruit_TinyUSB.h>
#include <Wire.h>
#include <Adafruit_LPS2X.h>
#include <Adafruit_Sensor.h>

Adafruit_LPS22 barometer;

void setup()
{
  // Enable the RAK19001 switched 3V3_S sensor rail.
  pinMode(WB_IO2, OUTPUT);
  digitalWrite(WB_IO2, HIGH);
  delay(500);

  Serial.begin(115200);
  delay(2000);

  Serial.println();
  Serial.println("=====");
  Serial.println("EMU-01 RAK12011 TEST");
  Serial.println("=====");

  Wire.begin();

  // RAK's current RAK12011 example uses I2C address 0x5D.
  if (!barometer.begin_I2C(0x5D))
  {
    Serial.println("RAK12011 NOT FOUND");
    Serial.println("TEST = FAIL");

    while (1)
    {
      delay(1000);
    }
  }

  barometer.setDataRate(LPS22_RATE_10_HZ);

  Serial.println("RAK12011 detected");
  Serial.println("TEST INITIALIZATION = PASS");
}

void loop()
{
  sensors_event_t temperature;
  sensors_event_t pressure;

  barometer.getEvent(&pressure, &temperature);

  Serial.print("RAK12011 temperature = ");
  Serial.print(temperature.temperature);
  Serial.println(" C");

  Serial.print("RAK12011 pressure = ");
  Serial.print(pressure.pressure);
  Serial.println(" hPa");

  Serial.println();
}
```

```
    delay(1000);  
}
```

Why keep `#include <Adafruit_TinyUSB.h>`? The lab RAK4631 BSP already showed that USB Serial failed to link without it. We keep the known-good compatibility fix while changing only the sensor layer.

Step 11.4 - Verify

Click:

✓ Verify

Expected successful compile pattern:

```
Sketch uses ... bytes (...) of program storage space.  
Global variables use ... bytes (...) of dynamic memory.
```

Logic:

```
Verify succeeds  
=  
library + sketch + RAK4631 BSP can build together
```

It does **not** yet prove the physical barometer.

Step 11.5 - Upload

Click:

→ Upload

Wait for:

Device programmed.

Then open:

```
Tools -> Serial Monitor  
baud = 115200
```

Step 11.6 - Expected output

A good result should look like:

```
=====  
EMU-01 RAK12011 TEST  
=====  
RAK12011 detected  
TEST INITIALIZATION = PASS  
RAK12011 temperature = ... C  
RAK12011 pressure = ... hPa
```

```
RAK12011 temperature = ... C
RAK12011 pressure = ... hPa
```

RAK specifies the RAK12011 pressure range as approximately 260-1260 hPa. Normal local atmospheric pressure should therefore be a plausible value in that supported range, not 0, nan, or an obvious error/sentinel value.

The temperature is the sensor's local/environment reading. Do not expect it to exactly match the later RAK1906/BME680 temperature because they are separate devices in different physical positions.

Step 11.7 - What counts as PASS?

```
RAK12011 detected           PASS
pressure is numeric/plausible PASS
temperature is numeric/plausible PASS
readings repeat without errors PASS
```

Current EMU-01 lab verification produced:

```
RAK12011 temperature = 26.49 C
RAK12011 pressure = 1015.66 hPa
```

This is a **functional PASS example**, not a permanent calibration baseline. The useful facts are that the sensor initialized, both fields were numeric, the pressure was physically plausible, and readings were returned without errors.

Unlike the light sensors, you do not need to force a dramatic physical change in atmospheric pressure. The initial goal is reliable detection and plausible pressure/temperature measurements.

Save the final firmware fields separately as:

```
barometer_pressure_pa
barometer_temperature_c
```

The RAK example reports pressure in hPa; the final payload can convert it to pascals with:

```
Pa = hPa x 100
```

Part 12 - Fourth physical sensor: RAK1906 / BME680 in Sensor E

After the RAK12011 passes, test the RAK1906 environmental sensor.

Permanent position:

```
Sensor E = RAK1906 / BME680
```

RAK documents that RAK1906 works in Sensor Slots A-F and uses I2C. In our fixed EMU-01 map it stays in Sensor E. The module measures four independent values:

```
temperature
humidity
pressure
gas resistance
```

Step 12.1 - Why test RAK1906 now?

We already know the shared I2C path works with three different devices. RAK1906 adds a multi-function environmental sensor and lets us verify the four fields that will later appear separately in the final payload.

```
known-good RAK4631 + I2C
  ↓
  RAK1906
  ↓
temperature
humidity
pressure
gas resistance
```

Do not merge these values with the RAK12011 fields. The two modules are separate physical sensors and will not necessarily report identical temperatures or pressures.

Step 12.2 - Install the required library

Open Arduino IDE Library Manager:

```
Sketch
-> Include Library
    -> Manage Libraries...
```

Search:

```
Adafruit BME680
```

Install **Adafruit BME680**. If Arduino asks to install **Adafruit Unified Sensor** or another required dependency, install the dependency too.

The test sketch needs:

```
#include <Adafruit_Sensor.h>
#include <Adafruit_BME680.h>
```

If Verify reports No such file or directory, stop and fix the library installation first. A missing header is still a software-build failure, not proof that the physical RAK1906 is bad.

Step 12.3 - Create the RAK1906 test sketch

Create a new sketch and paste:

```
#include <Adafruit_TinyUSB.h>
#include <Wire.h>
#include <Adafruit_Sensor.h>
#include <Adafruit_BME680.h>
```

```

Adafruit_BME680 bme;

void setup()
{
  // Enable the RAK19001 switched sensor power rail.
  pinMode(WB_I02, OUTPUT);
  digitalWrite(WB_I02, HIGH);
  delay(500);

  Serial.begin(115200);
  delay(2000);

  Serial.println();
  Serial.println("=====");
  Serial.println("EMU-01 RAK1906 TEST");
  Serial.println("=====");

  Wire.begin();

  // RAK's current RAK1906 example uses BME680 I2C address 0x76.
  if (!bme.begin(0x76))
  {
    Serial.println("RAK1906 NOT FOUND");
    Serial.println("TEST = FAIL");

    while (1)
    {
      delay(1000);
    }
  }

  bme.setTemperatureOversampling(BME680_OS_8X);
  bme.setHumidityOversampling(BME680_OS_2X);
  bme.setPressureOversampling(BME680_OS_4X);
  bme.setIIRFilterSize(BME680_FILTER_SIZE_3);
  bme.setGasHeater(320, 150);

  Serial.println("RAK1906 detected");
  Serial.println("TEST INITIALIZATION = PASS");
}

void loop()
{
  if (!bme.performReading())
  {
    Serial.println("RAK1906 reading FAILED");
    delay(5000);
    return;
  }

  Serial.print("RAK1906 temperature = ");
  Serial.print(bme.temperature);
  Serial.println(" C");

  Serial.print("RAK1906 humidity = ");
  Serial.print(bme.humidity);
  Serial.println(" %");

  Serial.print("RAK1906 pressure = ");
  Serial.print(bme.pressure / 100.0);
  Serial.println(" hPa");

  Serial.print("RAK1906 gas resistance = ");
  Serial.print(bme.gas_resistance);
  Serial.println(" ohm");

  Serial.println();
  delay(5000);
}

```

Why keep Adafruit_TinyUSB.h and WB_I02 = HIGH? Those are already proven parts of this EMU-01 lab setup: TinyUSB keeps USB Serial linking correctly with the installed RAK BSP, while WB_I02 enables the RAK19001

switched 3V3_S sensor supply.

Step 12.4 - Verify and Upload

Use the same gates:

```
Verify ✓
↓
Sketch uses ... bytes
Global variables use ... bytes
↓
software build PASS
↓
Upload →
↓
Device programmed.
↓
physical RAK4631 programming PASS
```

If Verify fails, fix the first compiler error before touching the hardware. If Verify passes but Upload fails, troubleshoot the COM/DFU layer instead of the BME680.

Step 12.5 - Open Serial Monitor

Open:

```
Tools -> Serial Monitor
```

Set:

```
115200 baud
```

Expected pattern:

```
=====
EMU-01 RAK1906 TEST
=====
RAK1906 detected
TEST INITIALIZATION = PASS
RAK1906 temperature = ... C
RAK1906 humidity = ... %
RAK1906 pressure = ... hPa
RAK1906 gas resistance = ... ohm
```

The exact numbers depend on the room and on sensor warm-up. For an initial functional pass, require numeric values and no repeated reading FAILED message.

Step 12.6 - BME680 burn-in and stabilization rule

This sensor is different from the light sensors and barometer because its gas-sensing element needs time to stabilize.

RAK's current RAK1906 instructions specify:

```
FIRST USE
read all values every 5 seconds
```

for at least 20 minutes

LATER POWER-UPS
allow approximately 2-3 minutes
for readings to stabilize

Why?

BME680 gas element powers/heats
↓
its response changes during warm-up
↓
early returned values may be valid communications
but are not yet a stable environmental baseline

Therefore separate two judgments:

FUNCTIONAL PASS
sensor detected + four numeric values

STABILIZED BASELINE PASS
required burn-in/stabilization completed
before freezing evidence or comparing runs

Do not treat the first gas-resistance number after boot as the final baseline.

Step 12.7 - What counts as PASS?

RAK1906 detected	PASS
numeric temperature	PASS
numeric humidity	PASS
numeric pressure	PASS
numeric gas resistance	PASS
repeated readings without error	PASS
20-minute first-use burn-in	REQUIRED once
2-3 minute later stabilization	REQUIRED before baseline capture

Current EMU-01 lab verification produced two consecutive samples:

RAK1906 temperature = 29.18 C
RAK1906 humidity = 51.21 %
RAK1906 pressure = 1011.12 hPa
RAK1906 gas resistance = 14166 ohm

RAK1906 temperature = 29.17 C
RAK1906 humidity = 51.19 %
RAK1906 pressure = 1011.12 hPa
RAK1906 gas resistance = 14641 ohm

Interpretation:

compile	PASS
DFU upload / "Device programmed."	PASS
RAK1906 communication	PASS
four measurement channels	PASS
repeat reading	PASS
first-use burn-in	COMPLETE (operator-marked)

The operator has marked the first-use burn-in gate complete. The gas-resistance value changing between early samples is not by itself a failure; later baseline captures should still allow the normal stabilization period before

comparing runs.

Do not require the RAK1906 pressure/temperature to exactly equal the RAK12011 values. They are separate sensor chips in different board positions and have different response/calibration behavior.

Final firmware fields remain separate:

```
environment_temperature_c
environment_humidity_percent
environment_pressure_pa
environment_gas_resistance_ohm
```

Part 13 - Fifth physical sensor: RAK12019 UV / LTR390 in Sensor C

After the RAK1906 functional test passes, move to the RAK12019 UV sensor.

Permanent position:

```
Sensor C = RAK12019 / LTR-390UV-01
```

RAK documents RAK12019 for Sensor Slots C-F, so Sensor C is valid for our fixed EMU-01 map. The module communicates over I2C and can operate in either ambient-light mode or ultraviolet mode. For the project payload, test **UV mode** because the final field is intended to be UV index.

Step 13.1 - Why test UV mode directly?

The RAK12019 can report both ALS and UV information. We already have two dedicated visible-light sensors, so the useful acceptance question is:

```
Can RAK12019 enter UVS mode?
↓
Can it return raw UVS data?
↓
Can the selected RAK driver return UVI using getUVI()?
↓
yes -> project may legitimately use uv_index
```

This resolves the earlier payload question: RAK's current example exposes `getUVI()` while the device is in `LTR390_MODE_UVS`, so the accepted firmware can use a UV-index field when it uses that driver/API path.

Step 13.2 - Install the RAK12019 library

Open Arduino IDE Library Manager:

```
Sketch
-> Include Library
-> Manage Libraries...
```

Search for:

```
RAK12019_LTR390
```

Install the library that provides:

```
#include "UVlight_LTR390.h"
```

Why? The library contains the LTR390 register configuration and the conversion logic used by RAK's example, including `getUVI()`.

If Verify reports:

```
fatal error: UVlight_LTR390.h: No such file or directory
```

then the failure is still at the compile/library layer. Do not reseal or replace the physical UV sensor yet.

Step 13.3 - Create the UV-mode test sketch

Create a new sketch and paste:

```
#include <Adafruit_TinyUSB.h>
#include <Wire.h>
#include "UVlight_LTR390.h"

UVlight_LTR390 uvSensor = UVlight_LTR390();

void setup()
{
  // Enable the RAK19001 switched 3V3_S sensor rail.
  pinMode(WB_IO2, OUTPUT);
  digitalWrite(WB_IO2, HIGH);
  delay(500);

  Serial.begin(115200);
  delay(2000);

  Serial.println();
  Serial.println("=====");
  Serial.println("EMU-01 RAK12019 UV TEST");
  Serial.println("=====");

  Wire.begin();

  if (!uvSensor.init())
  {
    Serial.println("RAK12019 NOT FOUND");
    Serial.println("TEST = FAIL");

    while (1)
    {
      delay(1000);
    }
  }

  Serial.println("RAK12019 detected");

  // Use UV mode, not ALS mode, for this project test.
  uvSensor.setMode(LTR390_MODE_UVS);

  if (uvSensor.getMode() == LTR390_MODE_UVS)
  {
    Serial.println("UV MODE = PASS");
  }
  else
  {

```

```

    Serial.println("UV MODE = FAIL");
}

uvSensor.setGain(LTR390_GAIN_3);
uvSensor.setResolution(LTR390_RESOLUTION_16BIT);

Serial.println("TEST INITIALIZATION = PASS");
}

void loop()
{
    if (uvSensor.newDataAvailable())
    {
        float uvi = uvSensor.getUVI();
        uint32_t rawUvs = uvSensor.readUVS();

        Serial.print("RAK12019 UVI = ");
        Serial.println(uvi);

        Serial.print("RAK12019 raw UVS = ");
        Serial.println(rawUvs);

        Serial.println();
    }

    delay(500);
}

```

This deliberately uses the same RAK12019 API names shown in RAK's current example: `init()`, `LTR390_MODE_UVS`, `setGain()`, `setResolution()`, `newDataAvailable()`, `getUVI()`, and `readUVS()`.

We keep `Adafruit_TinyUSB.h` because it is already a known-good USB-Serial compatibility requirement for this lab RAK4631 BSP.

Step 13.4 - Verify, Upload, and open Serial Monitor

Run the same three gates:

```

Verify ✓
  ↓
compile succeeds
  ↓
Upload →
  ↓
"Device programmed."
  ↓
Serial Monitor @ 115200

```

Expected startup:

```

=====
EMU-01 RAK12019 UV TEST
=====
RAK12019 detected
UV MODE = PASS
TEST INITIALIZATION = PASS

```

Then expect repeated UV readings such as:

```

RAK12019 UVI = ...
RAK12019 raw UVS = ...

```

The exact value depends heavily on the light source and actual UV content.

Current EMU-01 observation: zero UV indoors is INCONCLUSIVE, not FAIL

The current lab run successfully compiled, uploaded, and returned:

```
Device programmed.  
RAK12019 UVI = 0.00  
RAK12019 raw UVS = 0
```

Interpret this carefully:

compile/upload path	PASS
RAK4631 executes the UV sketch	PASS
RAK12019 driver returns numeric values	PASS
actual UV response	NOT PROVEN YET

0.00 / 0 can occur when the sensor is indoors or under a light source with little useful UV content. Do not call the module defective from this result alone, and do not call it a full sensor PASS yet.

Step 13.5 - Physical UV response test

Do **not** use an ordinary indoor LED lamp as the only UV test source. The purpose is to prove the sensor responds to a meaningful change in UV exposure.

Preferred test:

1. keep the current UV-mode sketch loaded
2. confirm Serial Monitor still prints UVI + raw UVS
3. move EMU-01 to a safe outdoor location with natural daylight
4. keep the RAK12019 optical surface unobstructed
5. wait for several fresh samples
6. record UVI + raw UVS
7. cover only the RAK12019 with an opaque object
8. record several samples
9. uncover it and record several more samples

Prefer direct outdoor daylight rather than testing only through a window because glazing can reduce ultraviolet transmission. Keep the electronics dry and do not leave the board where it can overheat. Do not use unsafe UV-C lamps or look directly into hazardous UV sources merely to force a response.

Expected logic:

```
outdoor / exposed  
↓  
UVS should become non-zero when meaningful UV is present  
↓  
cover sensor  
↓  
UVS/UVI should fall  
↓  
uncover sensor  
↓  
UVS/UVI should rise again
```

If UVS remains zero outdoors: use ALS mode as a diagnostic

The RAK12019 contains the same LTR390 device for both ambient-light and UV sensing. RAK's official example supports both modes. Temporarily change:

```
uvSensor.setMode(LTR390_MODE_UVS);
```

to:

```
uvSensor.setMode(LTR390_MODE_ALS);
```

Then, in the data-printing section, temporarily replace the UVI/UVS prints with:

```
Serial.print("RAK12019 Lux = ");  
Serial.println(uvSensor.getLUX());  
  
Serial.print("RAK12019 raw ALS = ");  
Serial.println(uvSensor.readALS());
```

Why this diagnostic helps:

```
ALS changes when cover/uncover changes light  
↓  
I2C + LTR390 + optical sensor response are alive  
↓  
problem is specifically UV exposure/mode/calculation sensitivity  
  
ALS also stays zero / does not react  
↓  
inspect Sensor C seating, optical obstruction, power, or module fault
```

After this diagnostic, restore LTR390_MODE_UVS because the project payload requires UV index rather than a third visible-light field.

Step 13.6 - What counts as PASS?

RAK12019 detected	PASS
UV mode confirmed	PASS
getUVI() returns numeric data	PASS
readUVS() returns numeric data	PASS
meaningful UV gives non-zero UVS	PASS
cover/uncover changes UV reading	PASS

The earlier indoor UVI = 0.00 / raw UVS = 0 observation remains the captured numeric example. The operator has now marked the RAK12019 physical-response acceptance gate **COMPLETE** for project tracking. No additional outdoor-response values were captured in this walkthrough, so do not invent or backfill measurements that were not recorded.

Final payload field:

```
uv_index
```

For payload v2 the encoded field remains `uv_index_x100`, so the final integrated firmware multiplies the accepted floating-point UVI by 100 and encodes it as the defined integer field. Do not substitute raw UVS counts into `uv_index_x100`.

Part 14 - Sixth physical sensor: RAK12023 + RAK12035 soil sensor on WisIO 1

The next bench-only test is the soil-moisture assembly:

```
WisIO 1 = RAK12023 connector module
          |
          └ RAK12035 soil probe
```

RAK12023 is the WisBlock IO connector board. The RAK12035 is the actual capacitive soil probe and reports **soil moisture** plus **soil temperature**. Only one RAK12035 probe is connected to a RAK12023 at a time.

Step 14.1 - Why calibration comes before the moisture test

Unlike the previous sensors, a raw soil-probe capacitance does not automatically mean a valid moisture percentage.

```
dry reference
  ↓
store as 0% reference
  ↓
wet reference
  ↓
store as 100% reference
  ↓
future capacitance readings
  ↓
converted to moisture percentage
```

RAK explicitly requires calibration before using the RAK12035 moisture value. The calibration values are saved on the RAK12035 sensor itself.

For this **functional bench verification**, use RAK's basic dry-air / water calibration. Before counted experiments, record the final project calibration method and, if higher measurement accuracy is required, calibrate against the selected dry-soil and water-saturated-soil references rather than treating air/water as a research-grade calibration.

Step 14.2 - Physical safety check

Before running the calibration:

```
[ ] RAK12023 is secured in WisIO 1
[ ] RAK12035 cable is firmly connected to RAK12023
[ ] main RAK19001 / RAK4631 electronics remain dry
[ ] probe is clean
[ ] container of clean water is stable and away from the base board
[ ] the probe's white immersion-limit line is visible
```

Important: RAK warns that the RAK12035 electronics are not fully waterproof. During the wet calibration, submerge only to the documented white line. Do not immerse the connector/cable electronics or the WisBlock base.

Step 14.3 - Install the soil library

In Arduino IDE:

```
Sketch
-> Include Library
    -> Manage Libraries...
```

Search:

```
RAK12035_SoilMoisture
```

Install the library that provides:

```
#include "RAK12035_SoilMoisture.h"
```

If Verify reports:

```
RAK12035_SoilMoisture.h: No such file or directory
```

that is a library-installation problem. The physical soil sensor has not been tested yet.

Step 14.4 - First run a read-status diagnostic before calibration

Do **not** write calibration values until the raw RAK12035 capacitance read has been proven healthy. The RAK12035 library returns a Boolean success/failure flag from `get_sensor_version()`, `get_sensor_capacitance()`, `get_sensor_moisture()`, `get_sensor_temperature()`, `get_dry_cal()`, and `get_wet_cal()`.

RAK's library explicitly says that when an I2C read returns `false`, the accompanying numeric value must be disregarded. A valid-looking temperature alone therefore does not prove that the capacitance channel is healthy.

Create a new sketch and paste this **read-only diagnostic**. It does not overwrite the stored dry/wet calibration values:

```
#include <Adafruit_TinyUSB.h>
#include <Wire.h>
#include "RAK12035_SoilMoisture.h"

RAK12035 soilSensor;

void setup()
{
  // Enable the RAK19001 switched sensor / IO power rail.
  pinMode(WB_IO2, OUTPUT);
  digitalWrite(WB_IO2, HIGH);
  delay(500);

  Serial.begin(115200);
  delay(2000);

  Serial.println();
  Serial.println("=====");
  Serial.println("EMU-01 RAK12035 READ DIAGNOSTIC");
  Serial.println("=====");

  Wire.begin();

  // begin() resets the RAK12035 and waits for it to respond.
  soilSensor.begin(true);
```

```

Serial.print("RAK12035 library I2C address = 0x");
Serial.println(soilSensor.get_sensor_addr(), HEX);

uint8_t version = 0;
bool versionOK = soilSensor.get_sensor_version(&version);

Serial.print("firmware read status = ");
Serial.println(versionOK ? "PASS" : "FAIL");

if (versionOK)
{
    Serial.print("RAK12035 firmware version = 0x");
    Serial.println(version, HEX);
}

uint16_t storedDry = 0;
uint16_t storedWet = 0;

bool dryOK = soilSensor.get_dry_cal(&storedDry);
bool wetOK = soilSensor.get_wet_cal(&storedWet);

Serial.print("stored dry read status = ");
Serial.println(dryOK ? "PASS" : "FAIL");
if (dryOK)
{
    Serial.print("stored dry calibration = ");
    Serial.println(storedDry);
}

Serial.print("stored wet read status = ");
Serial.println(wetOK ? "PASS" : "FAIL");
if (wetOK)
{
    Serial.print("stored wet calibration = ");
    Serial.println(storedWet);
}

Serial.println();
}

void loop()
{
    uint16_t capacitance = 0;
    uint16_t temperature = 0;
    uint8_t moisture = 0;

    bool capOK = soilSensor.get_sensor_capacitance(&capacitance);
    bool moistureOK = soilSensor.get_sensor_moisture(&moisture);
    bool temperatureOK = soilSensor.get_sensor_temperature(&temperature);

    Serial.print("capacitance status = ");
    Serial.print(capOK ? "PASS" : "FAIL");
    if (capOK)
    {
        Serial.print(" | value = ");
        Serial.print(capacitance);
    }
    Serial.println();

    Serial.print("moisture status = ");
    Serial.print(moistureOK ? "PASS" : "FAIL");
    if (moistureOK)
    {
        Serial.print(" | value = ");
        Serial.print(moisture);
        Serial.print(" %");
    }
    Serial.println();

    Serial.print("temperature status = ");
    Serial.print(temperatureOK ? "PASS" : "FAIL");
    if (temperatureOK)
    {

```

```

    Serial.print(" | value = ");
    Serial.print(temperature / 10.0);
    Serial.print(" C");
  }
  Serial.println();

  Serial.println("-----");
  delay(1000);
}

```

Why use this first?

```

read API returns PASS
+
raw capacitance is plausible and responsive
↓
communication layer proven
↓
THEN calibration is allowed

```

The current RAK library documents typical free-air capacitance in the low hundreds at 3.3 V, while published RAK calibration examples vary from probe to probe but are likewise in the hundreds. A raw value near the top of the 16-bit range (for example about 65500) is therefore not an acceptable calibration reference.

Step 14.4A - If the diagnostic passes, use RAK's official calibration example

After the read-status diagnostic shows successful capacitance reads with plausible raw values:

```

Arduino IDE
-> File
   -> Examples
       -> RAK12035_SoilMoisture
           -> RAK12035_Calibration

```

Use the official RAK calibration example for the actual dry-air and water calibration. Follow its prompts exactly:

```

dry probe in air
↓
100 calibration readings
↓
record dry value
↓
probe in water only to white line
↓
100 calibration readings
↓
record wet value
↓
values saved on RAK12035

```

The project keeps `Adafruit_TinyUSB.h` in custom lab sketches because the current RAK4631 BSP already proved that this include is needed for reliable USB Serial linking. If the stock RAK example produces the previously observed USB-Serial linker error, add `#include <Adafruit_TinyUSB.h>` at the top and Verify again.

Step 14.5 - Verify and Upload

Use the same gates:

```
Verify ✓  
↓  
compile succeeds  
↓  
Upload →  
↓  
Device programmed.
```

If compilation succeeds but Upload fails, troubleshoot the COM/DFU layer exactly as before; do not recalibrate or reseal the soil probe because an upload failure has not reached the sensor yet.

Step 14.6 - Configure Serial Monitor correctly

Open:

```
Tools -> Serial Monitor
```

Set:

```
baud rate = 115200  
line ending = Newline
```

The Newline setting matters because the calibration sketch waits for you to press **Enter** before each calibration stage.

Step 14.7 - Perform the dry calibration

When Serial Monitor shows:

```
STEP 1 - DRY CALIBRATION  
Keep the probe clean and completely in dry air.  
Press ENTER in Serial Monitor when ready.
```

Do this:

1. keep RAK12035 completely out of water/soil
2. make sure the sensing surface is dry
3. click the Serial Monitor input box
4. press Enter / send a blank line
5. do not touch the probe while 100 readings are collected

Expect:

```
Taking 100 dry readings...  
Dry calibration capacitance = <number>
```

The exact number is probe-specific; record it rather than comparing it to somebody else's value.

Step 14.8 - Perform the wet calibration

The sketch will next say:

STEP 2 - WET CALIBRATION
Submerge ONLY the allowed probe section up to the white line.
Press ENTER when ready.

Do this carefully:

1. place clean water in a stable container
2. move the container to the probe, not the powered base board to the water
3. lower ONLY the intended sensing section into the water
4. STOP at the documented white line
5. keep RAK12023, RAK19001, RAK4631, connectors, and cable electronics dry
6. press Enter
7. hold the probe still while 100 readings are collected

Expect:

```
Taking 100 wet readings...
Wet calibration capacitance = <different number>
Calibration values saved to RAK12035
CALIBRATION = COMPLETE
```

Why must the dry and wet values be different? The capacitive sensor is supposed to respond to the dielectric difference between dry air and a wet medium. If the two calibration values are essentially identical, do not accept the calibration.

Observed EMU-01 invalid calibration example

The first custom calibration attempt produced:

```
Dry calibration capacitance = 65500
Wet calibration capacitance = 65497
Soil capacitance = 65502
Soil moisture = 0 %
Soil temperature = 31.30 C
```

Historical status at this diagnostic point: INVALID CALIBRATION / SOIL NOT PASSED. SUPERSEDED by the later accepted dry=560, wet=328 calibration and final soil PASS recorded below.

Reasons:

```
dry and wet differ by only 3 counts
+
raw capacitance is near 65535
+
RAK documents normal capacitance in the hundreds
↓
these values must not be used as real calibration references
```

The earlier custom sketch also failed to check the Boolean success/failure result returned by the RAK12035 read APIs. Replace that workflow with the read-status diagnostic in Step 14.4 before recalibrating.

The bad calibration values already written to the RAK12035 are not permanent damage. Once the raw capacitance channel is healthy, rerun the official calibration and the correct dry/wet values will overwrite them.

The follow-up read-status diagnostic returned successful API transactions but still produced abnormal raw data:

```
capacitance status = PASS | value = 65494
moisture status = PASS | value = 66 %
temperature status = PASS | value = 28.80 C
```

```
capacitance status = PASS | value = 65498
moisture status = PASS | value = 100 %
temperature status = PASS | value = 28.80 C
```

Interpretation:

```
I2C/API transaction status      PASS
raw capacitance plausibility    FAIL
stored dry/wet calibration      INVALID (65500 / 65497)
reported moisture percentage    DO NOT TRUST
soil temperature channel        PLAUSIBLE, but soil assembly not yet accepted
```

The 66% -> 100% jump is expected from the invalid calibration span: the stored dry and wet references differ by only three counts, so tiny changes in the abnormal raw value can map to very large percentage changes. RAK's published examples use raw/calibration values in the hundreds and require dry/wet calibration references that meaningfully differ.

Step 14.8A - Isolate the abnormal raw-capacitance fault

Do **not** recalibrate and do **not** move to the rain test until the raw capacitance channel is plausible.

1. Close Serial Monitor.
2. Disconnect USB-C and remove all power from EMU-01.
3. Verify only one RAK12035 probe is connected to the RAK12023.
4. Reseat the RAK12035 cable at the RAK12023 connector; do not force the connector.
5. Verify the RAK12023 itself is fully seated and screwed into WisIO 1.
6. Inspect the probe cable and connector for bent contacts, partial insertion, damage, or moisture near the electronics end.
7. Reconnect USB-C.
8. Rerun the read-status diagnostic **without writing any calibration values**.
9. Observe at least 10 consecutive raw capacitance values with the probe clean and dry in air.

The EMU-01 reseat test then produced:

```
capacitance status = PASS | value = 563
capacitance status = PASS | value = 558
capacitance status = PASS | value = 561
capacitance status = PASS | value = 559
capacitance status = PASS | value = 561

temperature status = PASS | value = 29.30 C
moisture status = PASS | value = 100 %
```

Interpretation:

```
I2C/API transaction status      PASS
raw capacitance plausibility    PASS
raw dry-air stability           PASS
soil temperature                PASS / plausible
stored calibration              STILL INVALID (65500 / 65497)
reported moisture = 100%        IGNORE until recalibration
```

The change from ~654xx to stable 558-563 after reseating strongly indicates that the previous abnormal raw data came from the probe/connector seating path. The second RAK12035 probe is **not needed** for isolation unless the abnormal 654xx behavior returns.

Step 14.8B - Recalibrate now that the raw channel is healthy

The old 65500 / 65497 dry/wet values are still stored on the RAK12035, so the current 100% moisture output is not meaningful. Overwrite them with a new calibration.

Preferred Arduino path:

```
File
-> Examples
   -> RAK12035_SoilMoisture
       -> RAK12035_Calibration
```

If this lab BSP again needs the TinyUSB compatibility include, add:

```
#include <Adafruit_TinyUSB.h>
```

at the top before compiling.

Run the calibration in two stages:

```
DRY
probe clean and completely in air
↓
press ENTER
↓
collect calibration value
↓
expect a plausible value in the hundreds

WET
submerge only the sensing portion
up to the white line
↓
keep RAK12023 / connector / electronics dry
↓
press ENTER
↓
collect second calibration value
↓
expect another plausible value in the hundreds
```

Do **not** require wet capacitance to always be numerically higher or lower than dry. RAK's published examples show probe-specific values in both directions. The acceptance requirement is that the two stable calibration points are plausible and meaningfully separated, not that they follow a hard-coded direction.

The current official calibration run has now completed its wet stage and reported:

```
Calibration in water finished, average capacitance is 328
```

This is a plausible probe-specific wet calibration value and is clearly separated from the healthy dry-air raw readings observed immediately before calibration (558-563). However, record the **actual dry calibration value printed earlier in this same calibration run** before declaring the calibration complete. Do not silently substitute the earlier diagnostic raw range for the official saved dry calibration value.

Current calibration status:

raw channel after reset	PASS (558-563 in dry air)
dry calibration value	PASS = 560
wet calibration value	PASS = 328
calibration separation	PASS = 232 counts

stored dry/wet pair	PASS candidate = 560 / 328
wet-to-dry moisture response	HISTORICALLY PENDING HERE; LATER PASS / COMPLETE

After the example saves the values, verify the stored calibration and moisture response:

```
probe remains in wet calibration condition
  -> moisture should move toward the wet end

probe removed, cleaned, and dried
  -> moisture should move toward the dry end
```

If raw capacitance ever returns to ~654xx, stop calibration again and repeat the power-off/reseat check. Only if reseating no longer restores plausible values should Probe B be swapped onto the same RAK12023 for A/B isolation.

Step 14.9 - Observe the live measurements

After calibration the same sketch continuously prints:

```
Soil capacitance = ...
Soil moisture = ... %
Soil temperature = ... C
```

For a simple response check:

```
probe in/near the wet calibration condition
↓
moisture should be toward the wet end
↓
remove probe and dry the sensing surface
↓
allow readings to settle
↓
moisture should move toward the dry end
```

Do not demand an instant perfect 0% or 100% on every sample. The functional criterion is that calibration completes, the values are numeric, moisture responds in the correct direction, and temperature is plausible.

Step 14.10 - What counts as PASS?

RAK12035 responds / firmware version read	PASS
dry calibration value captured	PASS
wet calibration value captured	PASS
dry and wet references meaningfully differ	PASS
calibration values saved	PASS
soil capacitance numeric	PASS
soil moisture percentage numeric	PASS
soil temperature numeric/plausible	PASS
wet -> dry condition changes moisture correctly	PASS

Final payload fields remain separate:

```
soil_moisture_percent_x100
soil_temperature_c_x100
```

The Arduino library returns moisture as a percentage and temperature in tenths of a degree Celsius in RAK's current example. The final integrated firmware must explicitly scale these into the payload-v2 integer fields; do not

silently change units.

Part 15 - Seventh physical sensor: RAK12005 + RAK12030 rain sensor on WisIO 2

The next bench test is the rain / conductive-water detector:

```
WisIO 2 = RAK12005 rain detector module
          |
          └ RAK12030 sensing pad
```

RAK documents the RAK12005 as a digital electroconductive-liquid detector. The separate RAK12030 is the part that is intentionally exposed to water. The RAK12005 module, RAK19001 base, RAK4631, connectors, and the rest of EMU-01 must remain dry.

In the current EMU-01 fixed map the RAK12005 output is read on:

```
WB_IO6
```

RAK's current quick-start example also defines the RAK12005 water input as WB_IO6. The detector output goes HIGH when water is detected and LOW when the sensing pad is dry.

Step 15.1 - Why this test is simpler than the soil test

The rain sensor does not return a continuously calibrated moisture percentage. It is a binary detector:

```
RAK12030 dry
  ↓
RAK12005 comparator
  ↓
WB_IO6 LOW
  ↓
rain_wet = 0

RAK12030 wet
  ↓
conductivity across copper traces
  ↓
RAK12005 comparator
  ↓
WB_IO6 HIGH
  ↓
rain_wet = 1
```

The small potentiometer on the RAK12005 adjusts the digital detection threshold. Do not turn it unless the dry/wet test demonstrates that threshold adjustment is actually required.

Step 15.2 - Physical safety check

Before testing:

```
[ ] RAK12005 is fully seated and secured in WisIO 2
[ ] RAK12030 cable is firmly connected to RAK12005
```

```
[ ] RAK12030 sensing pad is clean and initially dry
[ ] RAK12005 electronics remain dry
[ ] RAK19001 / RAK4631 remain dry
[ ] water will be applied only to the exposed copper sensing pad
```

Use only a few drops of clean water for the bench test. There is no need to pour water over the board. After the wet test, wipe and dry the RAK12030 pad before checking the return-to-dry state.

Step 15.3 - No extra Arduino sensor library is required

RAK's current example reads the comparator output directly with `digitalRead(WB_I06)`, so this basic test does not need a separate RAK12005 Arduino library.

Keep the TinyUSB include because this lab RAK4631 BSP already proved that USB Serial requires it.

Step 15.4 - Create the rain detector test sketch

Create a new Arduino sketch and paste:

```
#include <Adafruit_TinyUSB.h>

#define RAIN_PIN WB_I06

void setup()
{
  // Keep the RAK19001 switched 3V3_S rail enabled for the assembled sensor set.
  pinMode(WB_I02, OUTPUT);
  digitalWrite(WB_I02, HIGH);
  delay(500);

  Serial.begin(115200);
  delay(2000);

  pinMode(RAIN_PIN, INPUT);

  Serial.println();
  Serial.println("=====");
  Serial.println("EMU-01 RAK12005 RAIN TEST");
  Serial.println("=====");
  Serial.println("DRY pad should report rain_wet = 0");
  Serial.println("WET pad should report rain_wet = 1");
  Serial.println();
}

void loop()
{
  int rainState = digitalRead(RAIN_PIN);

  Serial.print("RAK12005 digital state = ");
  Serial.println(rainState == HIGH ? "HIGH" : "LOW");

  Serial.print("rain_wet = ");
  Serial.println(rainState == HIGH ? 1 : 0);

  Serial.println();
  delay(1000);
}
```

Why use `WB_I06`? RAK's current RAK12005 example explicitly assigns its water-sensor input to `WB_I06`, and this matches the fixed EMU-01 pin map.

Why keep `WB_I02 = HIGH`? On the RAK19001, `WB_I02` controls the switched 3V3_S supply. Keeping it HIGH preserves the known-good powered state of the assembled sensor set during bench verification.

Step 15.5 - Verify, Upload, and open Serial Monitor

Run:

```
Verify ✓  
↓  
compile succeeds  
↓  
Upload →  
↓  
Device programmed.  
↓  
Tools -> Serial Monitor  
↓  
115200 baud
```

There is no external rain-sensor library to troubleshoot here. If Verify fails, focus first on board selection, TinyUSB/BSP, or syntax rather than the RAK12030 pad.

Step 15.6 - Dry-pad baseline

Start with the RAK12030 completely dry.

Expected output:

```
RAK12005 digital state = LOW  
rain_wet = 0
```

Observe at least five consecutive dry readings. A stable LOW state proves the detector is not falsely asserting rain while the pad is dry.

Step 15.7 - Wet-pad response

Apply a few drops of clean water directly across the exposed copper traces of the **RAK12030 sensing pad only**.

Do not wet the RAK12005 module or the base board.

RAK's current quick-start states that when water is present, the comparator output switches HIGH. Expected output:

```
RAK12005 digital state = HIGH  
rain_wet = 1
```

Observe at least five consecutive wet readings.

If the pad is clearly wet but the state remains LOW, first verify the RAK12030 cable and RAK12005 seating. Only then consider a small threshold adjustment using the RAK12005 potentiometer. Record any threshold adjustment because it becomes part of the device baseline.

Step 15.8 - Return-to-dry response

After the wet state is proven:

```
remove water
↓
wipe RAK12030 sensing pad
↓
allow copper traces to dry
↓
watch Serial Monitor
```

The output should return to:

```
RAK12005 digital state = LOW
rain_wet = 0
```

This wet-then-dry transition is important because a detector stuck HIGH would be useless for the final experiment even if it can detect the initial water application.

Step 15.9 - What counts as PASS?

RAK12030 initially dry	PASS
five stable dry readings = LOW / 0	PASS
water on pad changes state to HIGH / 1	PASS
five stable wet readings	PASS
pad drying returns state to LOW / 0	PASS
RAK12005/base electronics stayed dry	PASS

Final payload field:

```
rain_wet
```

Encode it exactly as:

```
0 = dry
1 = wet / conductive water detected
```

Do not invent an analog rainfall amount from this module. The RAK12005/RAK12030 path used here is a binary water/rain detector.

Part 16 - What has been proven after the bench sensor sequence

Current status after the completed tests and deferred UV stimulus is:

Arduino IDE / RAK BSP	PASS
USB / COM port	PASS
DFU bootloader upload	PASS
RAK4631 CPU execution	PASS
USB Serial	PASS
RAK19001 3V3_S sensor power	PASS
I2C basic operation	PASS
RAK1903 / OPT3001	PASS
RAK12010 / VEML7700	PASS
RAK12011 / LPS33HW	PASS
RAK1906 / BME680	PASS; burn-in gate COMPLETE (operator-marked)
RAK12019 / LTR390 UV	PASS; physical-response gate COMPLETE (operator-marked)

RAK12023 + RAK12035 soil	PASS; dry=560, wet=328, response gate COMPLETE (operator-marked)
RAK12005 + RAK12030 rain	PASS; compile/upload succeeded and operator confirmed responsive dry/wet behavior

EMU-01 SENSOR-TYPE COUNT = 7 / 7 COMPLETE.

All seven intended EMU-01 sensing types are now marked complete for project tracking. The seven types are OPT3001 light, LTR390 UV, LPS33HW barometer/temperature, BME680 environment, VEML7700 light, RAK12035 soil moisture/temperature, and RAK12030/RAK12005 rain detection. Where a completion was operator-declared rather than accompanied by new numeric output in this walkthrough, preserve that distinction in the evidence record instead of inventing measurements.

This still does **not** prove the B-copy sensors.

Part 17 - Error classification cheat sheet

Use the first error shown, not random hardware changes.

```
ERROR / SYMPTOM
- "No such file or directory" for a .h
  -> library/header problem
- undefined reference to Serial / Adafruit_USBD_CDC
  -> TinyUSB / BSP link problem
- Verify succeeds but Upload fails
  -> COM port / DFU / bootloader / USB problem
- "Device programmed." but Serial Monitor blank
  -> check port, baud 115200, reset, sketch execution
- Serial works but "Sensor not found"
  -> sensor power / I2C / slot / connector / device problem
- sensor prints values but physical stimulus does not change them
  -> wrong sensor being manipulated, stale/incorrect read,
    sensor placement, or sensor/driver problem
```

The principle is:

```
fix the first failing layer
before changing the next layer
```

Part 18 - Evidence to save

Record at minimum:

```
Arduino IDE version
RAKwireless BSP version
selected board = WisBlock Core RAK4631 Board
Windows COM port used during bring-up
sanity sketch compile result
sanity sketch upload result
RAK1903 library version
```

```
RAK1903 light response result
RAK12010 library version
RAK12010 light response result
RAK12010 Sensor-B/top-side light-clearance result
RAK12011 / Adafruit LPS2X library version
RAK12011 pressure/temperature result
RAK1906 / Adafruit BME680 library version
RAK1906 four-field result
RAK1906 first-use burn-in completion = COMPLETE (operator-marked; no new timed log captured here)
RAK12019_LTR390 library version
RAK12019 UV-mode/UVI/raw-UVS result
RAK12019 current captured observation = UVI 0.00 / raw UVS 0 indoors
RAK12019 physical-response acceptance gate = COMPLETE (operator-marked; no additional outdoor values captured here)
RAK12019 ALS diagnostic result if later needed
RAK12035_SoilMoisture library version
RAK12035 firmware version
RAK12035 dry calibration capacitance = 560 from the successful official calibration run
RAK12035 wet calibration capacitance = 328 from the successful official calibration run
RAK12035 moisture/temperature response gate = COMPLETE (operator-marked; no additional endpoint values captured here)
soil calibration method used for the final experiment
RAK12005 dry baseline = LOW / rain_wet 0
RAK12005 wet response = HIGH / rain_wet 1
RAK12005 return-to-dry response = LOW / rain_wet 0
RAK12005 threshold/potentiometer adjustment, if any
RAK12005 observed compile/upload = PASS on COM12
RAK12005 operator-confirmed responsive dry/wet behavior = PASS
any workaround required, including TinyUSB include
```

Do not store AppKeys or session keys here.

Suggested file:

```
chapter4-results/_device-baseline/sensors/arduino-first-bringup.txt
```

Continue

The EMU-01 A-set now has all seven intended sensing types marked complete for project tracking. Three acceptance items that previously remained open—RAK1906 first-use burn-in, RAK12019 physical-response verification, and the RAK12035 wet-to-dry response gate—were explicitly marked complete by the operator; no unrecorded numeric measurements are invented here.

The **RAK12005 + RAK12030 rain sensor on WisIO 2 is functionally PASS**: the sketch compiled, uploaded successfully on COM12, and the operator confirmed that the values respond to the pad's dry/wet condition. Proceed to the B-copy sensor verification on SEC-02.

Before moving any B-copy module, open [02b-rak19007-sec02-fixed-profiles.md](#). It freezes the SEC-02 positions as Profile A A=RAK1903-B, B=RAK12010-B, C=RAK12019-B, D=RAK12011-B, IO=RAK12023+RAK12035-B, followed—only after complete power removal—by Profile B A=RAK1906-B, B/C/D=EMPTY, IO=RAK12005+RAK12030-B. Do not use the EMU-01 RAK19001 A-F map on the RAK19007.

The broader checklist remains in:

[04-verify-all-sensors.md](#)

Official references

- RAK4631 Arduino BSP / nRF52 core: <https://github.com/RAKWireless/RAK-nRF52-Arduino>
- RAK1903 quick start: <https://docs.rakwireless.com/product-categories/wisblock/rak1903/quickstart/>
- RAK12010 quick start: <https://docs.rakwireless.com/product-categories/wisblock/rak12010/quickstart/>
- RAK12011 quick start: <https://docs.rakwireless.com/product-categories/wisblock/rak12011/quickstart/>
- RAK12011 datasheet: <https://docs.rakwireless.com/product-categories/wisblock/rak12011/datasheet/>
- RAK1906 quick start: <https://docs.rakwireless.com/product-categories/wisblock/rak1906/quickstart/>
- RAK1906 datasheet: <https://docs.rakwireless.com/product-categories/wisblock/rak1906/datasheet/>
- RAK12019 quick start: <https://docs.rakwireless.com/product-categories/wisblock/rak12019/quickstart/>
- RAK12019 overview: <https://docs.rakwireless.com/product-categories/wisblock/rak12019/overview/>
- RAK12023 quick start: <https://docs.rakwireless.com/product-categories/wisblock/rak12023/quickstart/>
- RAK12035 quick start: <https://docs.rakwireless.com/product-categories/wisblock/rak12035/quickstart/>
- RAK12005 quick start: <https://docs.rakwireless.com/product-categories/wisblock/rak12005/quickstart/>
- RAK12005 datasheet: <https://docs.rakwireless.com/product-categories/wisblock/rak12005/datasheet/>
- Adafruit TinyUSB: https://github.com/adafruit/Adafruit_TinyUSB_Arduino

Supporting runbook: 00-README.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Counted Test Execution

Use this folder only after the preparation relevant to the selected experiment scope has passed. The **full-stack** Chapter IV track requires the dedicated [sensor preflight](#) to produce `SENSOR_PREFLIGHT_STATUS=GO`. A deliberately narrower **LoRaWAN-only** track may begin earlier when `LORAWAN_TRACK_STATUS=GO`; that scoped exception authorizes only Execution 03 Part A (LoRaWAN authentication) and Execution 04 (LoRaWAN replay/spoofing). It does not authorize the final normal-operation baseline or any Fabric-dependent result.

The manuals are deliberately repetitive at the important points. That is intentional: every experiment tells you what state the system must be in, what to start, what to change, what to record, what makes a trial invalid, and how to restore the testbed afterward.

Dissertation-source rule

The current research-test authority is [../ZACARIAS-CHAPTER3-TEST-METRICS-SOURCE-OF-TRUTH.md](#), derived from the new chapters/Zacarias_Chapter3.pdf. Use that source for the **experiment questions, conditions, trial/run counts, durations, measurements, analysis rules, and stated acceptance/security criteria**. Disregard stale architecture/topology/service-placement prose in the chapter. Execute the required tests against the commissioned system that actually exists, using current runbooks and verified live state for implementation. Older Chapter III/IV/V drafts are historical context only when they conflict with the new source-of-truth.

Counted-test EMU-01 traffic profile

The counted Chapter IV runs intentionally use a **15-second normal-uplink cadence profile** to provide enough observations inside the defined 5-minute, 30-minute, and 2-hour windows. This is an experiment-only timing profile, not the accepted production scheduler.

Freeze these properties across every counted run:

```
payload = EMU-01 physical-sensor payload-v2, exactly 46 bytes
radio = plain AS923, OTAA, Class A
source identity = the same legitimate EMU-01 identity
normal counted cadence = 15 seconds
production cadence = 60-second local sampling + nominal 5-minute uplinks +/-15 seconds
SEC-02 = separate security fixture; never shares EMU-01 root/session keys
```

Before counted run 1, compile the tracked source with `EMU01_COUNTED_TEST_PROFILE=1`. Its **only intended behavioral difference from production is timing**. Require the boot banner

`EMU01_TRAFFIC_PROFILE=COUNTED_TEST_15S`, sample/normal intervals 15000 ms, and jitter 0. Archive source/config hash, exact build settings, BSP/toolchain versions, and compiled artifact SHA-256. Reuse that exact artifact for every 15-second counted run. After the series, rebuild/upload the default profile (`EMU01_COUNTED_TEST_PROFILE=0`), require the `PRODUCTION_5MIN` boot banner, and re-run the lightweight normal-path check.

The source sequence remains the independent delivery/ordering marker; physical sensor values are allowed to vary naturally.

Execution order

The existing numbered manuals are being reconciled to the new Zacarias Chapter 3 matrix. Until every affected manual is updated, the source-of-truth controls any conflicting count, condition, duration, metric, or outage definition.

1. [01-common-run-preparation.md](#) - freeze configuration, define run IDs, start/stop captures, collect logs, and classify VALID/INVALID runs.
2. [02-normal-operation.md](#) - **P1**: 3 x 30-minute normal-operation baseline runs. Formal PDR uses the ChirpStack acceptance boundary; exact source-to-database delivery is a separate metric.
3. [02a-fabric-performance-overhead.md](#) - **P2**: 1 tx/15 s, 1 TPS, 5 TPS, and 10 TPS for 5 minutes x 3 repetitions, plus the pre-registered matched no-Fabric controls needed for the overhead calculation.
4. [03-authentication-access-control.md](#) - **A1**: 90 attempts across LoRaWAN, MQTT, and Fabric identity/authorization.
5. [03a-fabric-endorsement-policy.md](#) - **A2**: 10 normal + 10 missing-endorsement violation + 10 post-restoration attempts, with ledger pre/post-state proof.
6. [04-replay-spoofing.md](#) - **S1**: 40 legitimate/replay/invalid-MIC attempts; attack frames count only after proven gateway reception.
7. **S2** - application-layer duplicate/replay remains BLOCKED_BY_METHODOLOGY: Chapter 3 lists it but does not define its trial count, exact procedure, metric formula, or pass rule. Do not invent a counted S2 result or silently substitute I3/database deduplication.
8. [05-data-integrity.md](#) - **I1/I2**: 40 application-layer and post-storage integrity trials.
9. [05a-blockchain-duplicate-overwrite.md](#) - **I3**: 10 baseline anchors + 10 duplicate attempts + 10 conflicting-overwrite attempts, with post-attempt ledger/hash proof.
10. [06-traceability.md](#) - **T1/T2**: 20 trials covering 60 records and DB-to-Fabric linkage.
11. [07-dos-flooding.md](#) - **F1/F2**: 18 five-minute runs plus five-minute recovery observation after every run.
12. [08a-internet-interruption-recovery.md](#) - **R1**: 3 x 2-hour external-Internet interruption runs (30 min normal + 60 min outage + 30 min recovery), implemented against the commissioned topology rather than the stale architecture.
13. [08b-fabric-consistency-reconciliation.md](#) - **R2**: 10 controls + 10 records created during Fabric unavailability, then reconcile those same 10 with query-before-resubmission evidence.
14. [09-results-and-completion.md](#) - calculate Chapter IV metrics from retained raw evidence and enforce the complete dataset checklist.

08-resilience-recovery.md remains only a legacy Fabric-endpoint-isolation reference. It is not the counted R1 Internet-interruption procedure.

Before starting the first counted test

For the complete Chapter IV stack, confirm the full-stack preflight file exists:

```
chapter4-results/_preflight/sensor/04-go-no-go/preflight-status.txt
```

and contains:

```
SENSOR_PREFLIGHT_STATUS=GO
```

For the scoped LoRaWAN-only security track, the full-stack file may remain NO-GO only for a downstream external dependency. Instead require `chapter4-results/_preflight/sensor/04-go-no-go/lorawan-track-status.txt` to contain `LORAWAN_TRACK_STATUS=GO`. That scoped GO authorizes only Execution 03 Part A (30 LoRaWAN authentication attempts) and Execution 04 (40 replay/spoofing attempts); it does not authorize the final normal-operation baseline or any result requiring a real Fabric submit/query/commit.

Then complete Execution 01 once for the selected scope, and run its short run-level preflight again before every experiment group.

The readiness gates and run-level precheck have different purposes:

```
full sensor preflight = prove the entire sensor/network/application/Fabric configuration is test-ready
LoRaWAN track gate   = prove the independent RF/LoRaWAN security path is test-ready
Execution 01          = start fresh evidence and prove the selected scope is healthy immediately before a
counted run
```

Do not reuse preflight packets/log windows as counted research data. Do not start a counted run because the UI merely "looks healthy." Use the readiness gate appropriate to the selected experiment scope plus a fresh known-good control uplink and working evidence capture immediately before the experiment group.

Universal run pattern

Every experiment uses this order:

```
A. PRECHECK
  Verify services, clocks, gateway, EMU-01, and evidence tools.

B. IDENTIFY
  Create unique RUN_ID / TRIAL_ID and record expected outcome before action.

C. CAPTURE
  Start source, server, gateway, resource, and experiment-specific evidence.

D. APPLY ONE CONDITION
  Baseline, invalid credential, replay, tamper, flood, or WAN loss.
  Do not combine conditions.

E. PROVE THE CONDITION HAPPENED
  Example: replay counts only when RAK5146 received the replay frame.

F. OBSERVE
  Record actual accept/reject/store/commit/recovery behavior.

G. STOP AND EXPORT
  Stop captures at the defined boundary and export raw evidence.

H. CLASSIFY
  PASS = expected behavior occurred and evidence is complete.
  FAIL = condition was validly applied but expected behavior did not occur.
  INVALID = the intended condition/evidence was not achieved; rerun separately.

I. RESTORE
  Return the testbed to the documented normal state before the next condition.
```

Standard result folder pattern

Use one directory per measured run or discrete trial group:

```
chapter4-results/<group>/<run_id>/
run-meta.txt
start-utc.txt
```

```
end-utc.txt
emu-01-source.log
server.log
gateway.log
docker-stats.csv
gateway-resource.csv
network-before.txt
network-after.txt
trial-results.csv          # when the experiment uses discrete trials
uplinks.csv                # when applicable
fabric.csv                 # when applicable
run-status.txt
notes.txt
```

Not every test needs every file, but never omit evidence required by its manual.

Universal trial fields

For every counted discrete trial retain at least:

```
trial_id
layer
test_condition
expected_result
actual_result
start_utc
end_utc
device_eui
frame_counter_or_event_key
test_sequence_when_applicable
gateway_received
application_reached
database_changed
fabric_tx_id
response_or_verification_time
trial_status = PASS | FAIL | INVALID
log_reference
notes
```

PASS, FAIL, and INVALID are different

PASS: the intended condition reached the layer being tested, the system behaved as expected, and evidence is complete.

FAIL: the intended condition reached the layer being tested, but the system behaved contrary to the expected result. Keep the failure; do not silently rerun until it disappears.

INVALID: the intended condition was not actually created or could not be proven. Examples: gateway never received the attack RF frame, load generator missed the required rate, required service crashed for an unrelated reason, timestamp correlation failed, or evidence capture was lost. Rerun the invalid attempt with a new trial ID and keep the invalid record in the audit trail.

Configuration discipline

Inside one repetition group, do not change:

```
firmware
container images
Node-RED flow logic
schemas
```

```
credentials used by the condition
AS923 plan
EMU-01 payload contract / 15-second interval
flood rates
WAN-interruption method
measurement interval
```

If a change is necessary, stop the group, document the change, repeat the **Execution 01 short run-level preflight**, and restart that repetition group with new IDs. Rerun the full sensor preflight only when the change affects the frozen sensor hardware/firmware, payload/codec, RF/AS923 path, or normal application mapping.

Result discipline

Keep raw files unchanged. Create calculated/cleaned summaries separately under `chapter4-results/summaries/`. Every Chapter IV value must be reproducible from retained raw evidence.

Canonical source: `deployment/technology-guides/18-rak4631-wisblock-sensor-firmware.md`. For any mutable production-status statement, check the dated baseline and verified live state.